



PDF Download
3731569.3764833.pdf
29 March 2026
Total Citations: 0
Total Downloads: 3738



Published: 12 October 2025

Citation in BibTeX format

SOSP '25: ACM SIGOPS 31st Symposium
on Operating Systems Principles
October 13 - 16, 2025
Seoul, Republic of Korea

Conference Sponsors:
SIGOPS

DL Latest updates: <https://dl.acm.org/doi/10.1145/3731569.3764833>

RESEARCH-ARTICLE

ORQ: Complex Analytics on Private Data with Strong Security Guarantees

ELI BAUM, Boston University, Boston, MA, United States

SAM BUXBAUM, Boston University, Boston, MA, United States

NITIN MATHAI, The University of Texas at Austin, Austin, TX, United States

MUHAMMAD FAISAL, Boston University, Boston, MA, United States

VASILIKI KALAVRI, Boston University, Boston, MA, United States

MAYANK VARIA, Boston University, Boston, MA, United States

View all

Open Access Support provided by:

Boston University

The University of Texas at Austin



ORQ: Complex Analytics on Private Data with Strong Security Guarantees

Eli Baum
Boston University
elibaum@bu.edu

Sam Buxbaum
Boston University
sambux@bu.edu

Nitin Mathai*
UT Austin
nitinm@cs.utexas.edu

Muhammad Faisal
Boston University
mfaisal@bu.edu

Vasiliki Kalavri
Boston University
vkalavri@bu.edu

Mayank Varia
Boston University
varia@bu.edu

John Liagouris
Boston University
liagos@bu.edu

Abstract

We present ORQ, a system that enables collaborative analysis of large private datasets using cryptographically secure multi-party computation (MPC). ORQ protects data against semi-honest or malicious parties and can efficiently evaluate relational queries with multi-way joins and aggregations that have been considered notoriously expensive under MPC. To do so, ORQ eliminates the quadratic cost of secure joins by leveraging the fact that, in practice, the structure of many real queries allows us to join records and apply the aggregations “on the fly” while keeping the result size bounded. On the system side, ORQ contributes generic oblivious operators, a data-parallel vectorized query engine, a communication layer that amortizes MPC network costs, and a dataflow API for expressing relational analytics—all built from the ground up.

We evaluate ORQ in LAN and WAN deployments on a diverse set of workloads, including complex queries with multiple joins and custom aggregations. When compared to state-of-the-art solutions, ORQ significantly reduces MPC execution times and can process one order of magnitude larger datasets. For our most challenging workload, the full TPC-H benchmark, we report results entirely under MPC with Scale Factor 10—a scale that had previously been achieved only with information leakage or the use of trusted compute.

CCS Concepts: • Security and privacy → Cryptography; • Information systems → Database query processing.

Keywords: secure analytics; multi-party computation

ACM Reference Format:

Eli Baum, Sam Buxbaum, Nitin Mathai, Muhammad Faisal, Vasiliki Kalavri, Mayank Varia, and John Liagouris. 2025. ORQ: Complex Analytics on Private Data with Strong Security Guarantees. In *ACM*

*Work done while at Boston University.



This work is licensed under a Creative Commons Attribution 4.0 International License.

SOSP '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1870-0/2025/10

<https://doi.org/10.1145/3731569.3764833>

SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25), October 13–16, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 32 pages. <https://doi.org/10.1145/3731569.3764833>

1 Introduction

Secure multi-party computation (MPC) [47] is a cryptographic technique that distributes trust across non-colluding parties, which perform a computation collectively. Thanks to its decentralized nature, it offers a robust solution that protects data both from individual parties and from powerful adversaries, who may have compromised a subset of the parties.

To avoid leaking information about the data, MPC programs are *oblivious*, i.e., they perform the same operations and memory accesses for all inputs of the same size. In practice, oblivious execution hides access patterns and the data distribution by generating worst-case outputs. For example, a secure MPC filter on a table with n records returns a table of the same size. This guarantees that the filter selectivity remains hidden from computing parties and external adversaries. However, hiding the data distribution for binary operators incurs a quadratic blowup in the worst case. An oblivious relational join on two input tables with n records will return the Cartesian product of size n^2 . The problem gets worse for large n and multi-way join queries, due to a cascading effect: in general, a naive oblivious evaluation of a query with k joins has $O(n^{k+1})$ time and space complexity.

Secure join queries have been extensively studied in the context of peer-to-peer MPC, where data owners also act as computing parties [10–12, 27, 50, 61, 62, 70, 71]. In this setting, performance can be improved by introducing controlled leakage [11, 12], offloading computation to trusted parties [70], or by applying optimizations tailored to specific data ownership schemes [10, 27, 50, 62, 71]. For example, SecretFlow [27] assumes the participation of exactly two data owners and Senate [62] expects each party to be a data owner who contributes an entire table to the analysis. Despite their particular differences, all of these works are restricted to a single (often custom) MPC protocol and threat model, offload expensive operations to data owners for plaintext execution in their trusted compute, and do not scale beyond a small number of participants.

To lift the restriction on the number of data owners and the need for trusted resources, MPC has also been deployed in the outsourced setting [2, 6, 17, 18, 26, 33, 43, 45, 51, 56, 63], where computation is performed by a small set of untrusted servers. Although promising, this approach makes oblivious join queries even more challenging, as servers never get access to plaintext data and cannot perform operations in the clear. Prior works on outsourced systems for relational analytics suggest that avoiding the cascading effect is inherently incompatible with strong security guarantees: Secrecy [45] ensures no information leakage but its join operator has quadratic cost, while Scape [33] reduces the complexity to $O(n \log^2 n)$ but leaks the join result size to the servers.

Core problem. Making secure analytics practical in the outsourced setting requires tackling a hard open problem: *how to efficiently evaluate queries that involve joins where both input tables may contain duplicates*. This is the crux of the cascading effect and becomes the norm when multiple data owners contribute data to a joint analysis. To evaluate such queries, all prior MPC works that support fully oblivious execution either compute the Cartesian product of the input tables (which is expensive), enforce a subquadratic upper bound on the join output (which silently drops joined records), or leak the join result size (which is insecure). In fact, the problem is not specific to MPC but manifests itself in the broader area of oblivious analytics. When it comes to joins on duplicate keys, even state-of-the-art approaches for trusted hardware either require an upper bound on the join result [22, 76] or introduce volume leakage [53].

This paper describes the design and implementation of ORQ, an *Oblivious Relational Query engine* that enables efficient analytics in the outsourced MPC setting with strong security guarantees. ORQ takes a holistic approach that considers the entire workload and supports a broad class of analytics without suffering from the limitations of prior approaches. We make the observation that all relational queries used in prominent MPC works (even those that include multi-way joins on potentially duplicate keys) produce *results whose size is worst-case bounded by the input size*; that is, there exists an upper bound $O(n)$ on the query output that is independent of the data distribution. By analyzing 31 real and synthetic workloads we collected from the standard TPC-H benchmark [67] and prior MPC works, we find that the cascading effect is avoidable in all of them, even in the queries that have been used to pinpoint the problem and motivate the need for leakage [11]. ORQ leverages the data-independent upper bound on the query result size to achieve fully oblivious tractable execution—entirely under MPC—with no leakage and without relying on trusted compute.

A new scalable approach to oblivious analytics. In a typical MPC scenario, data owners would only agree to participate in analyses that return aggregated results (to preserve their privacy), and, oftentimes, the aggregation function is

amenable to partial evaluation. Driven by this insight, ORQ’s core novelty is a technique that avoids materializing the Cartesian product of joins by decomposing and applying the aggregation eagerly (whenever possible) to bound the size of intermediate results. In practice, ORQ performs a series of composite join-aggregation operations resembling a MapReduce-style evaluation [59]. To achieve competitive performance, ORQ generalizes recent oblivious shuffling and sorting techniques [5, 60] to different MPC protocols, optimizes them for tabular data (to avoid redundant column permutations), and integrates them with a butterfly-style control flow [14, 29] (to further reduce communication rounds between parties). It evaluates all queries we have collected from prior MPC works with *asymptotic cost equal to that of sorting the input tables*: $O(n \log n)$ operations, $O(\log n)$ rounds, and $O(n)$ memory, where n is the total number of input records.

Contributions. We make the following contributions:

- We develop ORQ, a system that enables complex analytics on large private datasets using cryptographically secure MPC. ORQ provides a novel system runtime that employs data-parallel vectorized execution, a high-level dataflow API for composing secure analytics on tabular data, and an efficient communication layer that amortizes MPC costs in LAN and WAN settings.
- We design an oblivious join-aggregation operator that supports all join types (inner, outer, semi- and anti-join), a broad class of aggregation functions, and composition with itself or other operators (e.g., user-defined filters) to form *arbitrary* data pipelines. Using this operator and known transformations from relational algebra, ORQ can evaluate all queries we have collected from prior MPC works, without suffering from the quadratic blowup of intermediate results.
- We design all ORQ operators to make black-box use of MPC primitives (e.g., $+$, $\times$, div , $\oplus$, $\wedge$). As such, they can be easily instantiated with *any* MPC protocol to support use cases with different threat models and security requirements. To showcase generality, we instantiate ORQ with three protocols: two semi-honest (for honest and dishonest majorities) and one maliciously secure protocol in the honest-majority setting.
- We present a comprehensive experimental evaluation on diverse workloads, including the *complete* TPC-H benchmark (for the first time under MPC)¹ and nine queries from prior works inspired by real use cases. We show that ORQ significantly outperforms state-of-the-art relational MPC systems while also scaling complex analytics to one order of magnitude larger inputs.

Additionally, we contribute data-parallel vectorized implementations of oblivious shuffling, quicksort, and radixsort

¹ We do not yet support fixed-point arithmetic and substring operations.

that achieve better concrete performance than existing approaches. For oblivious sorting, we report results with up to half a billion rows—10× larger than the best published results [4, 5] (whose implementations are not publicly available). We have released ORQ as open-source at:

<https://github.com/CASP-Systems-BU/orq>

2 System overview

ORQ targets the outsourced setting in Figure 1 that has recently gained popularity in industry [2, 33, 43, 51, 56], academia [18, 21, 26, 45, 63], and non-profit organizations [69]. In our setting, *data owners* distribute *secret shares* of their data to a set of untrusted non-colluding *computing parties*, in practice, servers managed by different infrastructure providers. Computing parties execute a query on the input shares, following a specified MPC protocol, and send shares of the result only to the designated *data analysts*. ORQ currently supports three MPC protocols and can be instantiated with semi-honest or malicious security. We describe ORQ’s secret-sharing techniques and protocols in §2.3-2.4.

ORQ is designed for the typical outsourced setting with no access to a Trusted Compute Base (TCB) [33, 45], but can also be deployed on premises in scenarios where data owners want to participate in the computation [10, 62, 70] (and in that case it could use optimizations from prior work that leverage trusted resources). Naturally, ORQ also supports the scenario in Flock [40], where a single data owner wants to securely outsource some computation to the cloud.

2.1 Supported workloads

ORQ supports a rich class of relational workloads, allowing an arbitrary composition of the following operators to form query plans: SELECT, PROJECT, INNER_JOIN, (RIGHT / LEFT / FULL)-OUTER_JOIN, SEMI-JOIN, ANTI-JOIN, GROUP BY, DISTINCT, ORDER BY and LIMIT. It also supports all common aggregations (COUNT, SUM, MIN, MAX, AVG) and user-defined aggregations, which can be constructed with its secure primitives: addition, multiplication, bitwise operations, comparisons, and division with private or public divisor. ORQ does not impose any restrictions on the database schema, such as the existence of integrity constraints (e.g., primary or foreign keys), but analysts can leverage these constraints, if they exist, to improve execution performance.

What can ORQ compute efficiently? ORQ’s focus is on complex, multi-way join-aggregation queries that have so far been considered impractical in the outsourced MPC setting [6, 33]. Specifically, ORQ can efficiently compute acyclic conjunctive queries [72] that include: (i) only one-to-many joins, i.e., joins where at least one input has unique keys, or (ii) many-to-many joins (with duplicate keys in both inputs) as long as there exists a decomposable (or algebraic) aggregation function [30, 73], and any group-by keys appear in a single input table. Interestingly, all queries we have collected

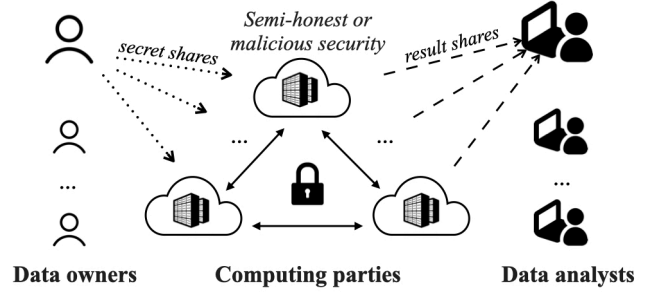


Figure 1. ORQ targets the typical outsourced setting that uses a small set of computing parties (whose number depends on the MPC protocol) to support any number of data owners and analysts.

from prior relational MPC works [10–12, 27, 45, 50, 62, 70, 71] fall in one of these two categories.

We note that many-to-many joins (also known as cross joins) are particularly common in collaborative data analysis because integrity constraints (e.g., the existence of unique join keys) may not be public. Moreover, even if integrity constraints are known, it is hard to enforce them outside MPC, unless data owners rely on a trusted third party or are willing to reveal some information about their data.

What can ORQ not compute efficiently? There are queries outside this class that are inherently difficult to perform under MPC, and for these queries ORQ falls back to an oblivious $O(n^2)$ join algorithm, like prior work. Examples of such queries (in SQL syntax) are: SELECT COUNT(*) FROM A, B, C WHERE A.X=B.Y AND B.Y=C.W AND C.W=A.Z (cyclic), or SELECT COUNT(*) FROM A, B WHERE A.X=B.Y AND A.W>B.Z, when A.X contains duplicates (aggregation across tables).

2.2 Programming model

ORQ users write queries in a dataflow-style API, similar to the ones provided by Apache Spark [74] and Conclave [70]. Relational operators are modeled as transformations on input tables and can be chained to construct complex queries. Users submit their queries to the ORQ engine, which compiles them into secure MPC programs for the computing parties.

Listing 1 shows an implementation of TPC-H Q3 in the ORQ API (see the benchmark specification [67] for the SQL definition). The query combines filters, joins, and aggregations to compute the shipping priority and total revenue of customer orders that had not been shipped as of a given date.

Filters (lines 5-7) expect logical predicates that are constructed with ORQ secure primitives (e.g., $\wedge$, $=$, $\neq$, $\geq$, $\leq$). Joins expect a key and an optional list of aggregation functions (each with an input and output column) that will be applied on the same key as the join. Line 13 joins tables C and O on column CustKey. The result is a new (anonymous) table that is then joined with table LI on column OrdKey, in line 14. The join propagates OrdDate and Priority values from the left input to the output by applying the function copy.

```

1 // Initialize C (Customers), O (Orders), LI (Lineitem)
2 ...
3
4 // Apply filters
5 C.filter("MktSegment" == SEGMENT);
6 O.filter("OrdDate" < DATE);
7 LI.filter("ShipDate" > DATE);
8
9 // Calculate the revenue
10 LI["Revenue"] = LI["Price"]*(100-LI["Discount"])/100;
11
12 // Joins and group-by with aggregation
13 auto RES = C.inner_join(O, {"CustKey"})
14   .inner_join(LI, {"OrdKey"}, {
15     {"OrdDate", "OrdDate", copy},
16     {"Priority", "Priority", copy}})
17   .aggregate({"OrdKey", "OrdDate", "Priority"},
18     {"Revenue", "TotalRevenue", sum})
19   .project({"OrdKey", "TotalRevenue",
20     "OrdDate", "Priority"})
21   .sort({"TotalRevenue", DESC},
22     {"OrdDate", ASC})
23   .limit(10);

```

Listing 1. TPC-H Q3 with the ORQ API

The aggregate operator (line 17) expects a list of columns as grouping keys ($\{"OrdKey", "OrdDate", "Priority"\}$) and a list of functions along with their input and output columns ($\{"Revenue", "TotalRevenue", sum\}$). ORQ can apply multiple aggregation functions in a single control flow (see §3.5) and can also update columns in place (e.g., $\{"C", "C", sum\}$) to reuse memory. ORQ provides a set of built-in aggregation functions (e.g., count, sum, min, max, and avg) but also allows users to define custom ones.

Our rationale for exposing a dataflow-style API in ORQ instead of a SQL interface was motivated by a recent critique of SQL [58]. We also found that multi-way join queries with nested sub-queries written with the ORQ API were often more concise and easier to understand than their SQL counterparts. For this paper, we implemented 31 queries of varying complexity in a few hours.

2.3 Secret sharing

ORQ protects data using *secret sharing* [65]. Given N computing parties, a secret ℓ -bit string s is encoded using ℓ -bit shares $s_1, s_2, \dots, s_N$ that individually are uniformly distributed over all possible ℓ -bit strings and collectively are sufficient to reconstruct the secret s . ORQ supports two types of secret sharing: *arithmetic* (for numbers) and *boolean* (for strings or numbers). In arithmetic sharing, the secret number s is encoded with N random shares such that $s = s_1 + s_2 + \dots + s_N \bmod 2^\ell$, where mod is used to handle overflow. In boolean sharing, the secret string s is encoded with N random shares such that $s = s_1 \oplus s_2 \oplus \dots \oplus s_N$, where $\oplus$ is the bitwise XOR. ORQ provides efficient MPC primitives to convert between the two representations without relying on data owners.

Data owners distribute secret shares to non-colluding computing parties, which in practice are ORQ servers running in different trust domains. Each computing party receives a strict subset of the shares (per secret) so that no single party

can reconstruct the original data. Given random shares, parties can execute a secure MPC protocol to jointly perform arbitrary computations on the shares—as if they had access to the secrets—and end up with shares of the result. Operations on shares break down into a set of low-level secure primitives (e.g., $+$, $\times$, $\oplus$, $\wedge$), which are more expensive than the respective plaintext primitives because they require communication: given two ℓ -bit shares, secure multiplication (for arithmetic shares) and bitwise AND (for boolean shares) require exchanging $O(\ell)$ bits between computing parties. Addition and bitwise XOR are local operations.

2.4 Threat model and security guarantees

Like in prior works [26, 45, 62, 70], data owners and analysts who want to use ORQ must agree beforehand on the computation to execute. The query and data schema are public and also known to the computing parties. To ensure data privacy, ORQ uses end-to-end oblivious computation and protects all input, intermediate, and output data—including the sizes of intermediate and final results—throughout the analysis.

Threat models. MPC protocols protect data against computing parties and external adversaries who may control T out of N computing parties, where $T < N$. ORQ can withstand adversaries who can be *semi-honest* (“honest-but-curious”) or *malicious*. A semi-honest adversary passively attempts to break privacy without deviating from the protocol (by monitoring the state of the parties it controls, e.g., via inspection of messages and access patterns). A malicious adversary actively attempts to break privacy or correctness and can deviate arbitrarily from the protocol for the parties it controls.

Supported protocols. ORQ operators are protocol-agnostic, that is, they use the underlying MPC functionalities ($+$, $\times$, $\oplus$, $\wedge$) as black boxes [20]. This way, they can be instantiated for any MPC protocol by simply replacing the corresponding functionalities. In its current version, ORQ supports three state-of-the-art protocols with different threat models and guarantees: (i) the semi-honest ABY protocol [23] for dishonest majorities ($T = 1, N = 2$), (ii) the semi-honest protocol by Araki et al. [3] for honest majorities ($T = 1, N = 3$), and (iii) the malicious-secure Fantastic Four protocol [19] for honest majorities ($T = 1, N = 4$).

Security guarantees. ORQ inherits the security guarantees of its underlying MPC protocols, ensures a fully oblivious control flow, and always operates over secret-shared data. It provides (i) *privacy*, which means that computing parties and adversaries do not learn anything about the data, and (ii) *correctness*, which means that all participants are convinced that the computation result is accurate. Our malicious-secure protocol provides security *with abort*, meaning that honest parties halt the computation if they detect malicious behavior (to protect data privacy). That said, ORQ can easily be amended to support robust execution if desired.

3 The ORQ approach to oblivious analytics

We first review oblivious primitives that we use as building blocks and we then describe ORQ’s sorting protocols in §3.2. We present ORQ’s core join-aggregation operator in §3.3 and its extensions in §3.4-3.6. In §3.7, we show a step-by-step example evaluation of a multi-way join query in ORQ.

Notation. We denote a column C in table T as $T.C$. For a row $r \in T$, we denote its C value as $r.C$. To simplify the presentation, we avoid secret-sharing notation and describe the control flow of operators as if they are applied to plaintext data. When saying “an operator is applied to table T ,” we mean that ORQ parties perform MPC operations on the secret-shared table T , and produce secret shares of the result.

Oblivious computation. We say a computation is oblivious if its control flow and observable side effects (such as data access patterns and timing) are input-independent, so that a computing party learns nothing about input, output, or intermediate values. Obliviousness prevents data reconstruction attacks [13, 31, 38, 39, 57] but has an inherent overhead compared to plaintext computation, as it requires transforming all data-dependent conditionals (if-then-else statements) into data-independent ones. For example, consider the expression if $a > b$ then $c = a$ else $d = b$. As written, a computing party (or adversary) could infer the relationship $a > b$, by observing whether memory location c or d was written to. Such expressions exist in relational analytics, e.g., in filters and CASE statements. To avoid leaking information about the data, ORQ transforms and evaluates this expression as shown below:

```
condition = (a > b)
c = (1 - condition) * c + condition * a
d = (1 - condition) * b + condition * d
```

Note that we now write to both memory locations, irrespective of the condition, and follow no data-dependent branches. In practice, a , b , c , and d are secret-shared values, and operators $*$, $+$, $-$ in the above listing correspond to secure functionalities provided by the underlying MPC protocol.

3.1 Oblivious building blocks

To hide true table sizes, ORQ tables (input, intermediate, or output) have a special *validity column* of secret-shared bits, encoding which rows are “valid” (1) or “dummy” (0). Dummy rows are those invalidated by oblivious operators and can also exist in input tables as a result of padding by data owners. Since valid bits are secret-shared, ORQ servers cannot distinguish real from invalid rows and operate on worst-case table sizes during the entire execution. Validity columns are never exposed to parties; they are managed by ORQ operators transparently. Invalid rows are masked and shuffled before opening to prevent leakage of “deleted” data.

ORQ provides efficient implementations of core oblivious operators for relational tables. An oblivious **filter** applies a

Protocol 1: AGGREGATION NETWORK (AGGNET)

Input : Table T with n rows, group key column K ,
function f , input column A , result column G
Result : Table T with one value in G per unique K

```

1  $r.G \leftarrow r.A$  // Copy input into result
2 for  $d = 1; d < n - d; d = d * 2$  do
3   for each pair of rows  $(r_i, r_{i+d}), 0 \leq i < n - d$  do
4      $b \leftarrow r_i.K == r_{i+d}.K$  // Compare keys
5      $g \leftarrow f(r_i.G, r_{i+d}.G)$  // Aggregate
6      $r_{i+d}.G \leftarrow \text{Mux}(b, r_{i+d}.G, g)$  // Update
```

predicate to each row in the input table and outputs a secret-shared bit that denotes whether the row passes the filter. Oblivious **deduplication** (the “distinct” operator) is applied to one or more columns treated as a composite *key*. It obviously sorts the table on these columns and then compares adjacent rows to mark the first occurrence of each distinct key with a secret-shared bit. Oblivious **multiplexing** is used in sorting, join, and aggregation operators. An arithmetic multiplexer $\text{Mux}(b, x, y)$ implements $b ? y : x$ by evaluating $(1 - b) \cdot x + b \cdot y$, where $b \in \{0, 1\}$ and x, y are the elements to multiplex (boolean multiplexers are defined similarly).

Oblivious **aggregation** identifies groups of records based on a (composite) key and applies one or more aggregation functions to each group, updating the input table in place. ORQ implements oblivious aggregation using sort followed by a butterfly-style network [14, 29, 37] which requires $O(n \log n)$ operations and messages, $O(\log n)$ communication rounds, and $O(n)$ space per party, where n is the cardinality of the input table. Protocol 1 shows the control flow of the aggregation, which resembles the Hillis-Steele network [34]. We provide a proof of correctness in Appendix C.2.

The algorithm expects a table T sorted on column K (the grouping key) and compares keys at distance d , while doubling the distance after every iteration. Each step applies an aggregation function f to pairs of input values and multiplexes the result (in-place) into column G , based on whether the respective records belong to the same group ($b = 1$). For simplicity, the pseudocode assumes a single key column K and a single aggregation function f . In practice, however, ORQ’s operator expects one or more key columns and can evaluate more than one aggregation function *on the fly* (on the same or different columns) by reusing b (line 4). In §3.3, we show how ORQ leverages this functionality to reduce the cost of joins followed by aggregation.

3.2 Sorting protocols for tabular data

ORQ sorts tables in place using Protocol 2 (TABLESORT). The protocol expects a table and a subset of columns (1 to k) to use as sorting keys. The strawman approach is to sort the entire table for each key, which would incur expensive communication under MPC. This strawman is commonly

Protocol 2: TABLESORT

Input : Table T with p columns $\{C_1, \dots, C_p\}$, integer $k \leq p$, orders $o = \{o_1, \dots, o_k\}$, $o_i \in \{\text{ASC}, \text{DESC}\}$

Output : Table T sorted on $\{C_1, \dots, C_k\}$

```

1  $\pi \leftarrow \mathbb{I}$  // Identity permutation
2 for  $i$  from  $k$  to 1 descending do
3    $t \leftarrow C_i$  // Temp copy of column  $i$ 
4    $\text{applyPerm}(t, \pi)$  // Permutation so far
5    $\pi' \leftarrow \text{sort}(t, o_i)$  // Sort temp column
6    $\pi \leftarrow \text{composePerm}(\pi, \pi')$  // Build iteratively
// Apply final permutation to all columns
7 for  $i$  from 1 to  $p$  do
8    $\text{applyPerm}(C_i, \pi)$ 

```

used in other MPC systems that sort multiple columns, such as Secrecy [45]. TABLESORT avoids this overhead as follows: it extracts sorting permutations (i.e., mappings from old to new positions) from key columns in line 5, composes them from right to left (line 6), and applies the final permutation to all columns of the table once at the end (lines 7-8). To extract sorting permutations from a sorted vector, we expand each input element with additional bits encoding its original position before sorting, and we extract these bits from the sorted result into a vector. The vector of extracted bits amounts to a sorting permutation. In practice, this happens on secret-shared data, and the resulting vector is a secret sharing of the mapping from old to new element positions (see Appendix B for more details).

Unlike prior works, TABLESORT uses different protocols to extract sorting permutations, depending on the key bitwidth. By default, it uses quicksort for large bitwidths (e.g., $\ell = 64$) and radixsort for small bitwidths ($\ell \leq 32$). Both algorithms rely on state-of-the-art oblivious shuffling [5, 60], which we generalize to work with all MPC protocols in ORQ. We briefly describe ORQ's sorting protocols below and provide further information on shuffling and sorting in Appendix A and B.

Oblivious Quicksort. Quicksort has a non-oblivious and recursive control flow that makes its naïve application to the MPC setting problematic in both efficiency and security. The typical recursive instantiation of the algorithm would incur $O(n)$ rounds of comparisons, which is prohibitively slow. To address this challenge, we devise an iterative control flow that applies all independent comparisons at a given recursion depth simultaneously, facilitating vectorization and reducing the number of rounds to $O(\log n)$. To guarantee security, we employ a *shuffle-then-sort* [4, 32] approach that avoids leaking information about the relative order of input data. We expand the n input elements with $\lceil \log n \rceil$ additional (secret-shared) bits to encode each element's index in the original table and guarantee that elements are unique. We then apply oblivious shuffling, which moves the input elements to random positions and enables parties to safely

execute the standard quicksort control flow by opening the (single-bit) result of each secure comparison in the clear to rearrange the elements. By obviously shuffling a vector of unique elements, the control flow of the comparisons in quicksort is independent of the input vector, so we are free to reveal the results of comparisons without compromising security. Indeed, prior work [4, 32] has proven that revealing these random bits does not leak any information about the data (and this should not be confused with one-bit leakage protocols, e.g., [75]).

Oblivious Radixsort. Radixsort sorts a vector one bit at a time, moving from the least to most significant bit. Bogdanov et al. [16] permutes the entire vector after each round of bit comparisons, while the later approach by Asharov et al. [5] composes the permutations bit by bit and applies them to the vector at the end, similarly to what we do in TABLESORT at the table level. For radixsort, we have found that a hybrid approach which combines the eager technique by Bogdanov et al. [16] with the shuffling primitives by Asharov et al. [5] reduces the number of rounds for a small increase in bandwidth and outperforms both protocols by up to 1.44 $\times$. ORQ uses a vectorized implementation of this hybrid approach.

3.3 A composite join-aggregation operator

Let L, R be two tables with dimensions $n \times p$ and $m \times q$, respectively. $L.V$ and $R.V$ are the special validity columns. Without loss of generality, we assume that L and R are joined on j common columns $K = \{K_1, \dots, K_j\}$, $0 < j \leq p, q$. We collectively refer to the columns in K (and their values) as the *join keys* $L.K$ and $R.K$. The join keys can be arbitrary columns and should not be confused with Primary Keys (PK) or Foreign Keys (FK). ORQ's join-aggregation operator is based on oblivious sort and an aggregation network. It is more general than a PK-FK join, in that it handles cases where rows in either input may not have matching keys on the other side. It is also not equivalent to a sort-merge join, in that it computes a join and an (optional) aggregation within the same oblivious control flow (to reduce MPC costs).

Overview of basic operator. We first describe the basic skeleton for a simple equality join that requires $L.K$ to be unique, but allows $R.K$ to contain duplicates. Later on, we discuss how to further generalize the operator to other join types, including joins with duplicate keys in both inputs. The operator joins L with R (denoted as $L \bowtie_K R$) to identify all pairs of rows from L and R with the same key K . The result is a new table O that includes all columns from R along with columns C from L . Given that keys in L are unique, the cardinality of table O is at most $m = |R|$, enabling ORQ to chain multiple such join operators while keeping memory bounded. If an aggregation function is provided, the operator also applies the function to each group of rows in O with the same key K . Protocol 3 shows the pseudocode and Figure 2 depicts the internal steps of the operator.

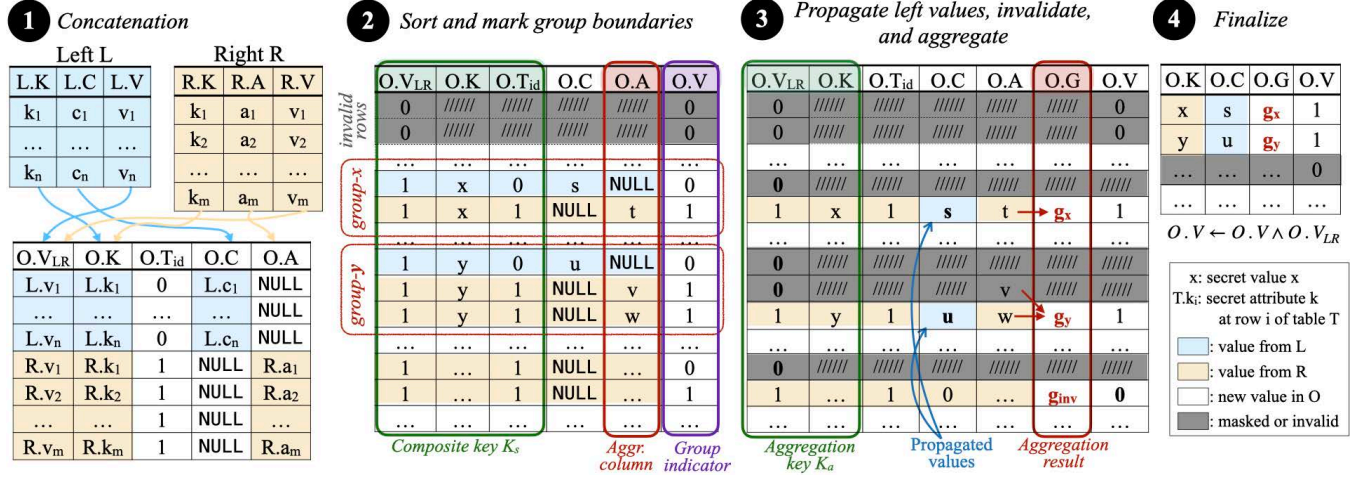


Figure 2. Steps of the basic ORQ operator. The input tables are concatenated and sorted to bring valid rows with the same join key K (e.g., x and y) next to each other. Values in $O.C$ are copied to matching rows from R . Values in $O.A$ are aggregated per key K and stored in $O.G$.

Step 1: Concatenation. In the first step, the operator initializes the output table O with $n + m$ rows and $p + q - 1$ columns from the two input tables. It combines the validity columns $L.V$ and $R.V$ into $O.V_{LR}$ and the join key columns $L.K$ and $R.K$ into $O.K$. Next, it appends an auxiliary column $O.T_{id}$ and initializes rows $[1 \dots n]$ with 0 and rows $[n + 1 \dots n + m]$ with 1, indicating the origin table (L and R , respectively) of each row. Finally, it expands the non-key columns from L and R with n (resp. m) NULL values and appends them to O . Figure 2 shows a concatenation example. For simplicity, we assume one key (K) and one non-key column per table (C and A resp.). The resulting table O has $p + q - 1 = 5$ columns.

Step 2: Sort and mark group boundaries. The operator sorts table O on the composite key $K_s = \{O.V_{LR}, O.K, O.T_{id}\}$ using the table sort protocol from §3.2. This way, valid rows with the same key K are clustered together, and each row from L becomes the first row of its respective K group. Each group of rows in O has at most one row from L and zero or more rows from R . Next, the operator calls oblivious distinct from §3.2 with key $K_a = \{O.V_{LR}, O.K\}$ to mark the first row of each group. It then flips the result bit of distinct and ANDs it with $O.V_{LR}$ to update the validity column $O.V$. The first row of each group now has $O.V = 0$.

Step 3: Propagate values from L , invalidate rows, and apply aggregation. In this step, the operator calls the AGGNET protocol from §3.1 to apply two functions: f_{copy} and f_{agg} . Function f_{copy} is an internal function that takes as input a pair of values from $O.C$ and simply returns the first one, i.e., $f_{copy}(x, y) = x$. It is used by ORQ to copy non-key values from L into R by obliviously populating the NULL values, and also to invalidate rows from R that did not match with any row from L . Function f_{agg} is a user-provided aggregation function. The operator applies f_{agg} to column A and stores its result in a new column $O.G$. The function f_{agg} can be one

Protocol 3: JOIN-AGGREGATION (JOIN-AGG)

Input : Tables L, R ; keys $K = \{K_1, \dots, K_j\}$, columns $C = \{C_1, \dots, C_k\}$ to propagate from L to R , an aggregation function f_{agg} to apply to column A

Result : Table O

// Step 1: Initialize table O

- $O \leftarrow \text{CONCAT}(L, R)$ // Schema shown in Fig. 2
- let $K_s = \{O.V_{LR}, O.K, O.T_{id}\}$, // Sorting keys
 $K_a = \{O.V_{LR}, O.K\}$ // Aggregation keys
- TABLESORT(O, K_s)
- $O.V \leftarrow O.V_{LR} \wedge \neg \text{DISTINCT}(O, K_a)$ // Mark groups
- AGGNET($O, \{K_a, C, f_{copy}\}$, // Copy C into R
 $\{K_s, V, f_{copy}\}$, // Propagate V
 $\{K_s, A, f_{agg}\}$ // Aggregate
- Step 4: Remove redundant rows and columns
- FINALIZE(O)

of the common aggregations from §2.1 or a custom function constructed as we explain in Appendix C.

Step 4: Finalize result. At this point, O has $n + m$ rows; however, we know that the actual output of the join contains at most m valid rows. In the final step, ORQ removes redundant columns (e.g., $O.V_{LR}, O.A, O.T_{id}$ in Figure 2) and trims all unnecessary rows which came from L . To remove these rows, we sort O on $O.V$ (in descending order), placing valid rows above invalid ones, and trim the last n rows from the sorted table. In practice, we observe that the trimming operation is cheap but not costless, even when using our single-bit radixsort protocol: if $n \ll m$, the overhead of sorting by $O.V$ may be higher than the improvement gained in subsequent operations with the trimmed table. To address this tradeoff,

we develop a heuristic that *automatically* estimates whether trimming pays off, based on the sizes of the input tables. For example, in our semi-honest honest-majority protocol, the operator trims when $9m < n \lg n \lg \ell$, where ℓ is the share representation bitlength. Using this (conservative) heuristic has improved end-to-end execution times modestly across all queries of §5 and bounds the maximum working table size in queries with many joins. For a detailed analysis, see Appendix C.

3.4 Generalizing to different join types

JOIN-AGG computes inner joins with equality predicates; however, its control flow can support different join types with only minor changes to the way we invalidate rows. All operators of this section require unique keys on the left input (one-to-many) like the basic operator. We describe how to handle duplicates in both inputs in §3.6. For all join types we discuss, we provide correctness proofs in Appendix C.

Semi-join. A semi-join $L \bowtie_K R$ returns all rows in L that matched with any row from R on K . ORQ implements the semi-join by simply running JOIN-AGG with the two input tables swapped. After sorting O , rows from R will now precede rows from L in all groups that contain rows from both tables. The operator will therefore invalidate (and later trim) all rows from R and will only invalidate those rows from L that do not match with any row from R (i.e., they appear at the beginning of a group). In all other cases, the validity of rows from L remains as is (equal to $O.V_{LR}$), matching the semantics of semi-join without any changes in Protocol 3.

Outer joins. A left-outer join ($L \bowtie^{\leftarrow}_K R$) returns all pairs of rows from the two inputs that have the same key K plus all remaining rows from L that did not match with any row from R . To support a left-outer join, we only need to replace line 4 in JOIN-AGG with the following line:

$$O.V \leftarrow O.V_{LR} \wedge \neg(O.T_{id} \wedge \text{DISTINCT}(K_a))$$

This will invalidate the first row of each group only if it comes from R . If the row comes from L , its $O.V$ is set equal to $O.V_{LR}$. This ensures that valid rows from L do not get invalidated, even if they have no matches in R , but unmatched rows from R will be invalidated as in the basic operator.

A right-outer join ($L \bowtie^{\rightarrow}_K R$) works the opposite way: it returns all pairs of rows from the two inputs that matched on K plus all remaining rows from R that did not match with any row from L . In this case, we only need to replace line 4 in Protocol 3 with $O.V \leftarrow O.V_{LR} \wedge O.T_{id}$. The line invalidates all rows from L , since they have $O.T_{id} = 0$, and leaves the validity of rows from R intact (equal to $O.V_{LR}$). A right-outer join does not need to propagate valid bits and therefore omits this step while executing the AGGNET protocol in line 5.

Finally, the full-outer join ($A \bowtie^{\leftarrow\rightarrow}_K B$) does not invalidate any rows from either input. In this case, the operator sets $O.V \leftarrow O.V_{LR}$ in line 4 of Protocol 3 and does not propagate

any valid bit, as in the right-outer join above. All outer joins return $n + m$ rows and never trim rows from O .

Anti-join. An anti-join $L \bowtie^{\neg} R$ returns all rows from L that do not match any row from R . ORQ implements the anti-join by executing Protocol 3 with the two input tables swapped and line 4 replaced with $O.V \leftarrow O.V_{LR} \wedge O.T_{id}$, as in right-outer join. However, here we must propagate valid bits from R to L , which is done by invoking f_{copy} . When a row exists in R , it will be sorted above the rows from L and invalidated. Then, its valid bit ($= 0$) will be copied down to all other rows, including those from L , in its group.

Theta-join. ORQ's operator can support one-to-many joins with θ predicates, as long as θ is conjunctive and contains one or more equality conditions, e.g., $L.\text{Name} = R.\text{Name} \wedge L.\text{Time} \leq R.\text{Time}$. In this case, ORQ executes Protocol 3 using the equality predicate(s) that bound the output size and reduces all other predicates in θ into oblivious filters.

3.5 Supporting aggregations

In line 5 of JOIN-AGG, join is combined with an aggregation in the same oblivious control flow (AGGNET). In fact, ORQ can evaluate multiple aggregation functions f_{agg} with keys K_a on any combination of columns from L and R . If the query includes aggregations with keys different than K_a , an additional invocation of TABLESORT followed by AGGNET is necessary. In case the join or aggregation key is a prefix of the other, we can sort on the largest prefix in line 3 of JOIN-AGG to avoid the extra TABLESORT call.

ORQ supports decomposable aggregation functions of the form $f_{\text{agg}}(X) = f_{\text{final}}(f_{\text{post}}(f_{\text{pre}}(X)))$, where f_{pre} is a partial aggregation function over a set X , f_{post} combines partial aggregates, and f_{final} generates the final result. Self-decomposable functions are those with $f_{\text{final}} = \mathbb{I}$, the identity function. Function f_{pre} must itself be self-decomposable, $f_{\text{pre}}((X, Y)) = f_{\text{post}}(f_{\text{pre}}(X), f_{\text{pre}}(Y))$. Examples include $\min$, $\max$, sum (where $f_{\text{post}} = f_{\text{pre}}$), and count (where $f_{\text{post}} = \text{sum}$). Note that self-decomposability implies commutativity and associativity [35], which are necessary for AGGNET correctness. Decomposable aggregations enable efficient incremental computation and data-parallelism, and have been studied extensively in the context of sensor networks [52] and dataflow systems [73]. In ORQ, we employ similar ideas in oblivious computation to bound the size of intermediate results, as we explain next.

3.6 Supporting joins on duplicate keys

We can use JOIN-AGG to handle many-to-many joins (with duplicate keys on both sides), as long as the join is followed by a decomposable aggregation and the aggregation keys belong to one input table (see §2.1). This ensures that the worst-case cardinality of the output table is equal to the cardinality of the table with the aggregation key(s).

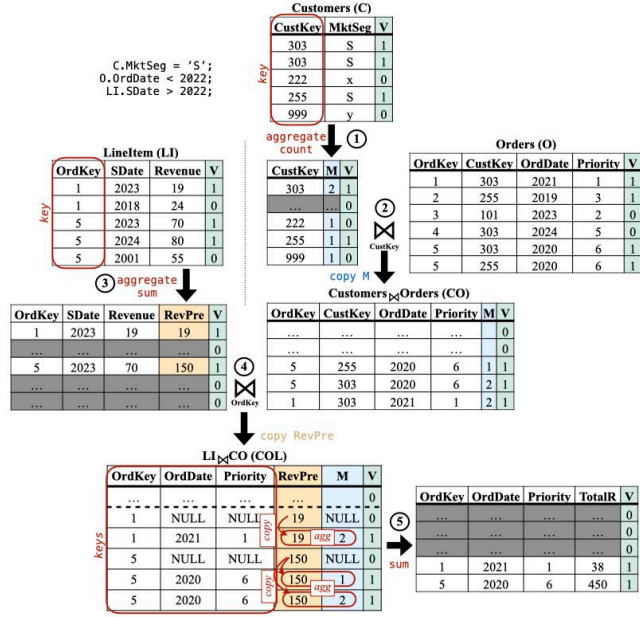


Figure 3. Step-by-step oblivious evaluation of TPC-H Q3 assuming no PK-FK constraints (all input tables have duplicate join keys).

Decomposable functions are amenable to partial evaluation; therefore, we can apply them without having to materialize the Cartesian product of the two input tables. As such, we apply f_{pre} before the join, f_{post} after the join, and, optionally, f_{final} to compute the final result. Let K_j and K_a be the join and aggregation keys, respectively. ORQ applies function f_{pre} to one of the input tables to compute a partial aggregation on K_j . The result is a table with unique join keys that can now be joined with the second table using the basic operator. Function f_{post} is evaluated on the output table O with aggregation key K_a .

As an example, consider a join $L \bowtie_{K_j} R$ on K_j with duplicate keys in both input tables followed by $f_{\text{agg}} = \text{count}$ that outputs the number of rows in the join result. We can obliviously compute the aggregation without a quadratic blowup by first evaluating function $f_{\text{pre}} = \text{count}$ on L to count the number of rows per key K_j (multiplicity). Then, we can join L with R using JOIN-AGG and apply $f_{\text{post}} = \text{sum}$ to add the multiplicities. The idea also applies to acyclic queries with multi-way joins like in plaintext query evaluation [36].

3.7 Putting it all together

Figure 3 shows an example with many-to-many joins, using a generalization of Q3 (see Listing 1), where we allow duplicates in all columns. That is, we assume no public knowledge of the PK-FK constraints in the TPC-H specification of this query. We omit filter operations to keep the example simple.

To evaluate $C \bowtie_{\text{CustKey}} O$, we first apply a pre-aggregation to C that computes the multiplicity of each CustKey using Protocol 1. The pre-aggregation makes CustKey in C

unique and stores multiplicities in a new column M . Next, we join C with O on CustKey using Protocol 3 to produce a new table CO . The second join $LI \bowtie_{\text{OrdKey}} CO$ is evaluated similarly: we first apply a pre-aggregation to LI that computes the Revenue per OrdKey and stores it in column RevPre. Then, we compute the join using Protocol 3 and apply the post-aggregation to sum the product $\text{RevPre} * M$ per key $K_a = \{\text{OrdKey}, \text{OrdDate}, \text{Priority}\}$ using Protocol 1. Listing 2 shows the implementation using the ORQ API.

```

1 // Pre-aggregate and apply first join
2 auto CO = C.aggregate({"CustKey"},
3   {"M", "M", count});
4   .inner_join(O, {"CustKey"},
5   {"M", "M", copy});
6 // Pre-aggregate and apply second join
7 auto COL = LI.aggregate({"OrdKey"},
8   {"Revenue", "RevPre", sum});
9   .inner_join(CO, {"OrdKey"},
10  {"RevPre", "RevPre", copy});
11 // Apply post-aggregation
12 COL["TotalR"] = COL["RevPre"] * COL["M"];
13 auto RES = COL.aggregate({"OrdKey", "OrdDate", "Priority"},
14   {"TotalR", "TotalR", sum});

```

Listing 2. The example of Figure 3 implemented in ORQ

4 System implementation

We have implemented ORQ from scratch in C++. Each computing party runs a software stack that includes (i) an MPC layer with vectorized implementations of secure primitives, (ii) a library of oblivious relational operators, (iii) a custom communication layer, and (iv) a data-parallel execution engine. Users can configure the share size 2^ℓ ($\ell = 64$ by default), the MPC protocol, and the sorting protocol (quicksort by default), according to their application needs.

We use libOTe [64] to produce random oblivious transfers, from which we generate OLE (Oblivious Linear Evaluation) correlations and Beaver triples. For permutation correlations, we use the technique by Peceny et al. [60]. For random number generation, we use AES from libsodium [46].

Protocol-agnostic secure operator abstractions. All relational operators, sorting, multiplexing, and division with private denominator are protocol-agnostic. The protocol-specific primitives are the functionalities defined by each MPC protocol (e.g., $\times$, $\wedge$, etc.), division by public denominator, and two shuffling primitives that we use to generate and apply sharded permutations. All other shuffle primitives (composing, inverting, and converting permutations) are protocol-agnostic. See also Supplementary A.

Vectorization and data parallelism. ORQ uses a columnar data format that allows for efficient vectorized execution. Recall that each secure multiplication and bitwise AND operation incurs a message exchange between parties. With vectorization, ORQ can efficiently group independent messages within operations and exchange them in the same network call. All low-level primitives and oblivious operators in ORQ are vectorized to work this way.

Each ORQ server runs as a single process with the same (configurable) number of threads τ . Execution within a server is data-parallel: a coordinator thread assigns tasks to workers, which operate on disjoint partitions of the input vector. ORQ's communication layer establishes a static mapping between workers, so that thread with id t communicates only with thread t in other servers, $0 \leq t < \tau$.

Communication layer. In ORQ, we implement a TCP-based communicator, tailored to I/O-intensive MPC execution. The communicator manages network sockets via a configurable number of dedicated threads. A high number of communication threads can significantly improve performance in WAN settings, where multiple parallel connections between parties can be used to leverage available bandwidth.

Worker threads interact with communication threads using lock-free ring buffers to avoid blocking computation. Communication threads are decoupled from worker threads and process ring buffers in a round-robin fashion. Data copies are avoided both ways; worker threads pass offsets in vectors they operate on, and the communicator directly pulls from (on send) and pushes to (on receive) these vectors.

5 Experimental evaluation

Our experimental evaluation is structured into four parts:

ORQ's performance on complex analytics. In §5.2, we present a comprehensive performance evaluation of ORQ in LAN and WAN with three MPC protocols. Our results demonstrate that ORQ provides excellent performance across the board and can compute complex multi-way join queries on millions of input rows within minutes.

Comparison with state-of-the-art. In §5.3, we compare ORQ with the following state-of-the-art systems:

- (i) Secrecy [45], a relational MPC framework targeting the 3-party outsourced setting with semi-honest security and no leakage. Secrecy is the only open-source relational system for this setting.
- (ii) SecretFlow [27], a relational MPC framework targeting the 2-party peer-to-peer setting with semi-honest security. SecretFlow is the only system we are aware of that scales some TPC-H queries to millions of rows per table, albeit by leaking information to parties.
- (iii) MP-SPDZ [42], a general-purpose MPC compiler used in many prior works. While MP-SPDZ does not support relational analytics, we compare with its sorting implementation, as sorting is the most expensive operator in relational queries.

ORQ provides up to $827\times$ lower query latency than Secrecy and its secure sorting implementations are up to $5.9\times$ and $189.1\times$ faster than the ones provided by SecretFlow and MP-SPDZ, respectively. We also achieve modest speedups over SecretFlow on queries with joins (up to $1.5\times$), despite the fact that SecretFlow introduces leakage while ORQ does not.

Scalability evaluation. In §5.4, we show that ORQ's operators scale to hundreds of millions of rows under all protocols. When configured with our most expensive protocol for malicious security, ORQ sorts 2^{29} rows in less than 2 hours.

We report additional results on ORQ's performance in a **geodistributed setting** in Appendix E, where we show that ORQ can be practically deployed over the Internet.

5.1 Evaluation setup

We use c7a.16xlarge AWS instances with Ubuntu 22.04.5 LTS and gcc 11.4.0, in two configurations: (i) LAN is an unconstrained setting in the us-east-2 (Ohio) region, with a maximum bandwidth of 25Gbps and a 0.3ms round-trip time (RTT), (ii) WAN is a restricted setting in us-east-2, with symmetric RTT of 20ms and 6Gbps bandwidth (as measured between us-west-2 and us-east-1 with 16 parallel iperf3 streams). Unless otherwise specified, ORQ is configured with 16 compute threads and the communicator uses 4 connections in LAN and 16 connections in WAN.

Workloads. We use 31 queries for evaluation. Our most comprehensive benchmark is the full set of TPC-H queries [67], implemented with the ORQ dataflow API. We replace floating point values with equivalent integral values and pattern matching expressions (`X like 'Y%'`) in six queries with oblivious (in)equality. To perform fully private averages, we implement a *non-restoring division* circuit inspired by the hardware literature [49]. Various prior works have implemented selected TPC-H queries [11, 27, 45, 62], but to our knowledge, we are the first to report performance numbers for *all* TPC-H queries (22 in total) entirely under MPC. We also implement all other queries we could find in prior relational MPC works [10–12, 27, 45, 50, 62, 70, 71]:

- *Aspirin*, *C. Diff*, *Password*, *Credit Score*, *Comorbidity*, and *SecQ2*, used in Secrecy [45], Conclave [70], and Senate [62], among others.
- *Market Share* from Conclave.
- *SYan*, Example 1.1 from Wang et al. [71].
- *Patients*, a 3-way join query used in Shrinkwrap [11] to showcase the cascading effect (which ORQ avoids).

Additionally, we implement the five peer-to-peer variations of TPC-H queries in SecretFlow [27], denoted as *S1-S5*. To validate correctness, we also implemented all queries in SQLite [66].

Inputs. In all experiments, we use $\ell = 64$ -bit shares by default. However, because our sorting protocols pad inputs to guarantee uniqueness and compute permutations, sorting is actually performed over 128-bit shares. We standardize a notion of query size: in the TPC-H specification [68], input tables grow according to the *scale factor* (SF) parameter, where SF1 corresponds to an (approximately) 1GB database. The smallest table (*Supplier*) has $10k \cdot SF$ rows, while the largest (*Lineitem*) has $6M \cdot SF$ rows. TPC-H queries have

total input sizes between $810k \cdot SF$ (Q11) to $8.5M \cdot SF$ (Q9) rows, and average $5.8M \cdot SF$ rows. For non-TPC-H queries, we accordingly define input sizes with $\approx 5M$ rows per SF.

Protocols. We label results according to the MPC protocol: **SH-DM** (semi-honest, dishonest majority) for ABY [23], **SH-HM** (semi-honest, honest majority) for Araki et al. [3], and **Mal-HM** (malicious-secure, honest majority) for Fantastic Four [19]. In all *SH-DM* experiments, we report online time, as in prior works [27].

5.2 Performance on complex analytics

In this experiment, we run ORQ at Scale Factor 1 (SF1) on the full workload, which includes queries of varying complexity. Queries like Q5, Q7, Q8, Q9 and Q21 are expensive, involving 4-7 joins each, with Q21 also performing a self-join on the largest input table, *LineItem*. Other queries include multiple filters, semi-joins, outer joins, group-by aggregations, distinct, and order-by operations, covering a wide range of query patterns. Q6 is the least expensive query, as it does not require sorting.

Figure 4 shows the end-to-end times in LAN and WAN and summarizes the result statistics. Overall, ORQ computes complex multi-way join queries on millions of input rows in a few minutes. The most expensive query, Q21, calls the sorting operator 12 times and, under malicious security, completes in 42 minutes over LAN. In the same setting, ORQ executes all other queries from prior work in under 10 minutes. Our results also demonstrate the effectiveness of ORQ’s vectorization and message batching. In the WAN environment, we see $1.2\times$ – $6.9\times$ higher execution times for a $75\times$ higher RTT.

5.3 Comparison with state-of-the-art systems

In this section, we compare ORQ with three state-of-the-art MPC frameworks on relational queries and oblivious sort.

Comparison on queries. We first compare ORQ with Secrecy [45], the only open-source relational MPC system without leakage that operates in the outsourced setting. We configure ORQ with the SH-HM protocol, which is the one used by Secrecy, and we execute all queries from the Secrecy paper, using the maximum input size they report [45, Fig. 4].

Figure 5 (left) shows the results. For the most expensive queries (Aspirin, Q4, and Q13) that perform a join or semi-join, ORQ achieves $478\times$ – $760\times$ lower latency. Password, Credit, Comorbidity, and C.Diff include group-by and distinct operators. For these queries, ORQ’s optimized sorting protocols also improve latency by $17\times$ – $42\times$. Q6 does not have any join or sorting, yet ORQ outperforms Secrecy by $3\times$. The significant improvements come from better asymptotic complexity of our join and sorting algorithms. Secrecy uses an $O(n^2)$ join and an $O(n \log^2 n)$ bitonic sort, while ORQ evaluates join-aggregation queries in $O(n \log n)$. Also, Secrecy’s runtime is single-threaded.

	TPC-H		Other	
	Median	Max	Median	Max
SH-DM LAN	3.8	15.0	1.6	3.1
WAN	13.5	52.7	6.2	11.7
SH-HM LAN	4.4	17.4	2.0	3.7
WAN	10.9	41.4	4.8	9.1
Mal-HM LAN	10.9	42.3	4.9	8.3
WAN	27.1	108.4	11.9	21.7

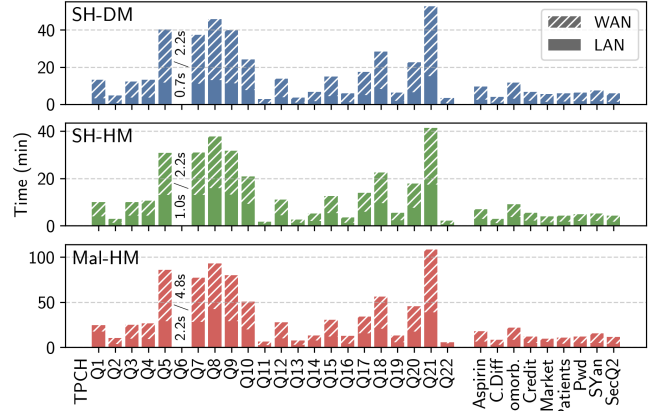


Figure 4. Query execution time (min) at SF1 in LAN (solid) and WAN (hatched). Bars are overlapped, not stacked. TPC-H queries shown on the left and other queries on the right. Q6 times annotated.

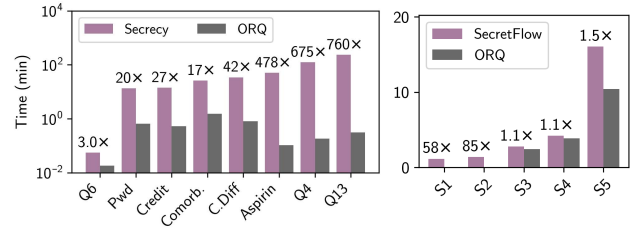


Figure 5. ORQ query times compared to Secrecy and SecretFlow. Secrecy operates in the outsourced setting without leakage. SecretFlow is a peer-to-peer system that offloads operations to the data owners’ trusted compute and leaks the result of the join to parties.

Next, we compare with SecretFlow [27], a recent relational MPC framework that operates in the peer-to-peer setting. To our knowledge, SecretFlow is the only open-source system of this type that scales TPC-H-based queries to millions of input rows per table. However, we emphasize that SecretFlow *leaks which rows match* to the parties, while ORQ does not introduce any leakage. Nevertheless, we include this comparison to show that ORQ can achieve competitive performance even in a peer-to-peer setting. We configure ORQ with the SH-DM protocol that uses the same underlying MPC primitives as SecretFlow (ABY [23]), and we allow both systems to perform operations in the data owners’ trusted compute, otherwise SecretFlow’s optimizations are not applicable.

Figure 5 (right) shows results for the five queries used in the SecretFlow paper, run on 16M input rows per table. ORQ

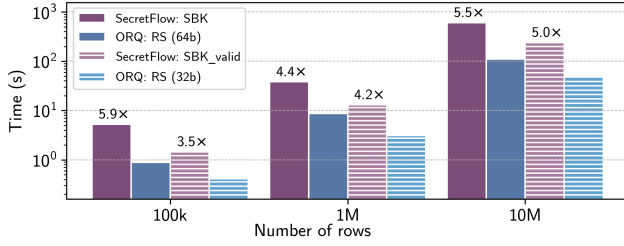


Figure 6. Performance of oblivious sort in SecretFlow and ORQ.

delivers superior performance in all cases, while protecting all data and intermediate result sizes. It achieves $58\times - 85\times$ lower latency for simple queries without joins (S1, S2) and $1.1\times - 1.5\times$ lower latency for join queries that also include aggregation (S3, S4) and oblivious group-by (S5). Despite SecretFlow’s leakage, ORQ still wins on performance due to its more efficient sorting and fused join-aggregation operator. Both systems use radixsort, but SecretFlow cannot leverage parallelism. In contrast, ORQ’s operators are vectorized and data-parallel.

Comparison on oblivious sort. We now compare ORQ’s oblivious radixsort against two publicly available state-of-the-art implementations. In Figure 6, we compare our SH-DM radixsort with SecretFlow’s SBK (64-bit radixsort) and SBK_valid protocols. SBK_valid is a modified radixsort that relies on data owners locally mapping plaintext data to use at most $\lceil \log_2 n \rceil$ bits, where n is the input size ($n = 17, 20, 24$ bits in this experiment). Even though it favors the baseline, we compare SBK_valid to our 32-bit radixsort and SBK to our 64-bit protocol. We see that ORQ is up to $4.4\times$ and $5.5\times$ faster when sorting 1M and 10M elements, respectively.

Our next comparison is with MP-SPDZ [42], a state-of-the-art general-purpose MPC library that provides secure sorting implementations for all MPC protocols supported by ORQ. We compare with its radixsort implementation, which is its most efficient sorting algorithm. Figure 7 shows the results. We vary the input size by powers of two, starting from 2^{16} elements and increasing the input size until MP-SPDZ either runs out of memory or crashes: up to 2^{22} elements with SH-DM, 2^{25} with SH-HM, and 2^{20} with Mal-HM. In the SH-DM and Mal-HM settings, ORQ’s radixsort is $189\times$ and $134\times$ faster than the corresponding MP-SPDZ implementations. Under the SH-HM protocol, which is heavily optimized in MP-SPDZ, ORQ achieves a $8.5\times$ speedup when sorting 2^{24} elements, however, MP-SPDZ could not complete the experiment for larger inputs. The large performance improvements over MP-SPDZ come from data-parallelism; although MP-SPDZ supports parallelism and advanced vectorization, it does not parallelize sorting.

Bandwidth consumption. We also evaluate bandwidth consumption in ORQ and the baselines. When compared to Secrecy, ORQ exhibits up to two orders of magnitude lower

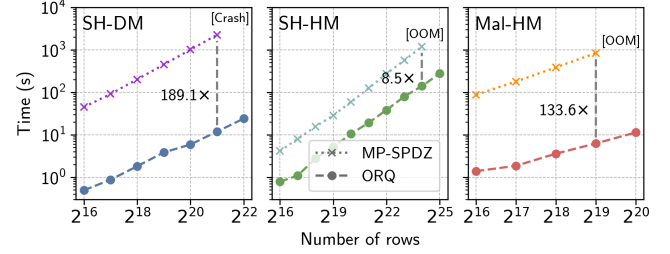


Figure 7. Performance of oblivious radixsort protocols in MP-SPDZ and ORQ. We run MP-SPDZ until it crashes or runs out of memory.

bandwidth consumption, due to our asymptotic improvements in sorting and join. Likewise, our sorting implementations achieve $1.6\times$ lower bandwidth than MP-SPDZ SH-HM and more than an order of magnitude improvement against MP-SPDZ SH-DM and Mal-HM. ORQ also achieves a $1.8\times$ lower bandwidth than SecretFlow in sorting. In the case of queries, SecretFlow has a lower bandwidth than ORQ, as it leaks matching rows in its join operator and performs subsequent operations locally. We report detailed results in Appendix D.

5.4 ORQ scalability

In this section, we show that ORQ can deliver practical performance for complex analytics on large inputs, without sacrificing security. To the best of our knowledge, we are the first to report results at this scale, for end-to-end MPC execution without leakage or the use of trusted parties.

TPC-H at SF10. We repeat the experiment of §5.2 on the TPC-H benchmark at Scale Factor 10. In this setting, queries operate on inputs of 58M rows, on average. The most expensive query, Q21, has a total input of size 75M rows and performs two of the largest joins, sorting tables with 120M rows each time.

Figure 8 plots the ratio of execution times at SF10 over SF1 for the SH-DM protocol in LAN. Assuming that query latency is dominated by sorting and aggregation, we would expect the runtime to scale as $O(n \log n)$. If the queries consisted *only* of ideal sorts, then theoretical scaling would be $10 \log(10n) / \log n \approx 11.5$, when $n \approx 5.8M$ (SF1). Overall, this is indeed the trend we see in Figure 8, with some outliers. In practice, queries are more complex and consist of operations with different costs. Some overheads are sublinear: for example, oblivious division is round-constrained for most inputs, so scaling appears lower for queries with division, like Q22. Other queries suffer slightly suboptimal scaling, due to the power-of-two padding required in AGGNET. For example, Q12 aggregates over a table of size 7.5M at SF1, which is padded to $2^{23} \approx 8M$ rows. However, at SF10, this same aggregation occurs on a table of size 75M, which now must be padded to $2^{27} \approx 134M$ rows, $16\times$ larger than the table at SF1.

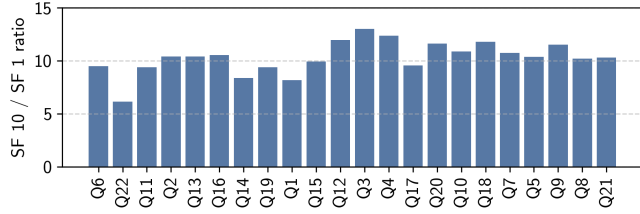


Figure 8. Ratio of TPC-H query execution times at SF10 over SF1 for the SH-DM protocol in LAN. The 22 queries are sorted from left to right by increasing SF10 latency.

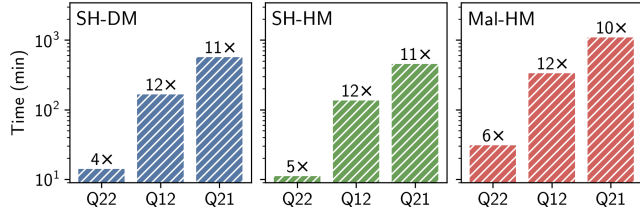


Figure 9. ORQ performance at SF10 in WAN. Bars are labeled with the scaling ratio compared to SF1 in the same environment. Q21 is the most expensive query in the TPC-H benchmark across protocols.

Next, we evaluate ORQ’s scalability in WAN, using the two outlier queries mentioned above, Q22 and Q12, and the most expensive query, Q21. Figure 9 shows the results for all protocols, where each bar is annotated with the scaling ratio compared to SF1. ORQ’s scaling behavior in this environment is consistent with our previous findings. Under malicious security, Q22 completes in 31 minutes and Q21 in 18 hours.

Scalability of sorting protocols. In the previous experiment, we showed that ORQ provides practical performance on complex queries that repeatedly sort tables with over 100M rows. We now stress ORQ’s oblivious sorting protocols further and evaluate their performance on even larger inputs. We run radixsort and quicksort in LAN with 32 threads, increasing the input size from 2^{20} to 2^{29} elements. Figure 10 shows the results. The slowest protocol, Mal-HM radixsort, can sort 134 million ($n = 2^{27}$) elements in about 35 minutes, while the fastest protocol, SH-HM quicksort, sorts 537 million ($n = 2^{29}$) in just over 70 minutes.

We find that quicksort and radixsort are competitive across settings. Quicksort is consistently faster under the malicious protocol, while radixsort is slightly faster in the SH-DM setting. Overall, quicksort (ORQ’s default sorting protocol) scales to larger inputs in our experimental setting, as radixsort has higher memory demands: it requires computing and storing $\ell + 2$ secret-shared permutations of length n .

6 Related Work

There is a large body of work on systems for secure computation. In this section, we only focus on works that are directly related to ORQ.

Relational MPC. Most systems and protocols for secure relational analytics target *peer-to-peer* settings and propose

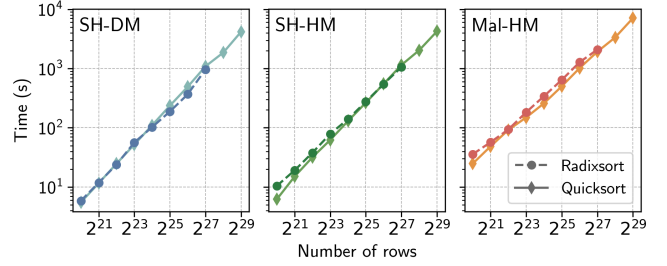


Figure 10. Scaling ORQ’s oblivious sorting protocols to very large inputs. Quicksort scales to larger inputs than radixsort, which has higher time and memory requirements for permutation generation.

techniques tailored to a small, fixed number of semi-honest data owners, who also act as computing parties: either two [10–12, 27, 61, 71] or three [50, 70]. In such settings, performance can be improved by splitting the query into (i) a plaintext part that data owners compute locally (e.g., filters, hashing, sorting, and grouping), and (ii) an oblivious part that is executed under MPC (e.g., joins, aggregations). Some of these works report results for a small subset of TPC-H queries on a maximum input size of 10M rows per table [27] (including rows processed in plaintext). Senate [62] proposes a custom malicious-secure circuit decomposition protocol that supports any number of parties and provides strong security against dishonest majorities. Even though the codebase is not publicly available, published results indicate that the protocol’s scalability is limited: the paper reports performance for a subset of TPC-H queries with 16k input rows (Scale Factor ~ 0.004), and three queries from §5 (Pwd, Comorb., and Credit) with up to 80k rows.

Contrary to ORQ, join optimizations in the above works rely on the assumption that each party owns one or more input tables. For example, they can improve performance in a scenario where a hospital and an insurance company wish to identify common patients with high premiums, but they are not effective (or do not even apply) in cases where multiple hospitals contribute to the patients table. ORQ’s operators are designed for the more general outsourced setting, hence, they are independent of the number of participants and data schema. Moreover, while circuit decomposition techniques pay off in early stages of complex queries, later stages still need to be evaluated under MPC by all parties. As a result, ORQ could also be used in the peer-to-peer setting to speed up the oblivious (and by far most expensive) part of complex queries. Other systems for outsourced MPC like Scape [33] (not publicly available) and Secrecy [45] report results for queries with up to two joins and maximum input sizes of 2M rows (for joins) and 8M rows (for other operators). Sharemind [15] (also not publicly available) reports results for queries with a single join and up to 17M input rows.

Oblivious join operators. Several works from the cryptography community study individual join operators in the outsourced setting, building from research into private set

intersection (PSI). Mohassel et al. [55] propose efficient one-to-one joins with linear complexity, but rely on the LowMC library that has been cryptanalyzed [48]. For one-to-many joins, Krastnikov et al. [44], Badrinarayanan et al. [8], and Asharov et al. [6] propose protocols based on sorting. They report results with up to 1M rows per input table and do not evaluate complex queries like in ORQ. These operators have the same asymptotic costs as our join-aggregation operator, but are tailored to 3-party computation protocols [6, 8] or target Trusted Execution Environments (TEEs) [44]. None of these works support many-to-many joins without suffering from the limitations we described in §1.

State-of-the-art systems for relational analytics in TEEs [22, 53, 76] also provide oblivious join and group-by operators. To protect access patterns from side-channel attacks, join and aggregation in these systems rely on bitonic sort, which requires $O(n \log^2 n)$ operations. Early systems perform the grouping using fast parallel scans [76], while recent approaches rely on prefix sums [53]. These TEE-based systems perform the join and the group-by one after the other. Consequently, they either leak intermediate result sizes [53], resort to worst-case padding [22], or require users to provide an upper bound on the query result size [76]. In contrast, ORQ uses a *composite* join-aggregation operator, with oblivious quick-sort and an aggregation network, that requires $O(n \log n)$ operations in total. The techniques used by ORQ in the MPC context can also be used to achieve oblivious execution in enclaves, and this is an exciting direction for future work.

7 Conclusion

We have presented ORQ, a scalable, secure framework for evaluating complex relational analytics under MPC. We instantiate ORQ in multiple threat models, contribute a unified join-aggregation operator to the literature, and implement state-of-the-art sorting protocols. We deploy ORQ to obliviously execute queries with multi-way joins larger than any prior work. As presented, ORQ requires data analysts to translate queries into our dataflow API; future work includes integrating ORQ with an automatic query planner. Cryptographic advances in oblivious sorting will also directly lead to performance improvements in ORQ.

Acknowledgments

We thank our shepherd, the anonymous reviewers and artifact evaluators, and Sakshi Sharma for their constructive feedback that substantially improved the paper. We also thank Selene Wu for implementing low-level optimizations in the ORQ runtime. This work was developed in part with computing resources provided by the Chameleon [41] and CloudLab [24] research testbeds, supported by the National Science Foundation. Final results and evaluation used Amazon Web Services clusters through the CloudBank project

(National Science Foundation Grant No. 1925001). This material is based upon work supported by the National Science Foundation under Grant No. 2209194, REU supplement awards No. 2432612 and 2326580, by the DARPA SIEVE program under Agreement No. HR-00112020021, and by a gift from Robert Bosch GmbH.

References

- [1] Navid Alamati, Guru-Vamsi Policharla, Srinivasan Raghuraman, and Peter Rindal. 2024. Improved Alternating-Moduli PRFs and Post-quantum Signatures. In *Advances in Cryptology – CRYPTO 2024: 44th Annual International Cryptology Conference, CRYPTO 2024, Santa Barbara, CA, USA, August 18–22, 2024, Proceedings, Part VIII* (Santa Barbara, CA, USA). Springer-Verlag, Berlin, Heidelberg, 274–308. doi:10.1007/978-3-031-68397-8_9
- [2] Apple and Google. 2021. Exposure Notification Privacy-preserving Analytics (ENPA). https://covid19-static.cdn-apple.com/applications/covid19/current/static/contact-tracing/pdf/ENPA_White_Paper.pdf. [Online; accessed April 2025].
- [3] Toshinori Araki, Jun Furukawa, Yehuda Lindell, Ariel Nof, and Kazuma Ohara. 2016. High-Throughput Semi-Honest Secure Three-Party Computation with an Honest Majority. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)* (Vienna, Austria), 805–817. doi:10.1145/2976749.2978331
- [4] Toshinori Araki, Jun Furukawa, Kazuma Ohara, Benny Pinkas, Hanan Rosemarin, and Hikaru Tsuchida. 2021. Secure Graph Analysis at Scale. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, Republic of Korea) (CCS '21). Association for Computing Machinery, New York, NY, USA, 610–629. doi:10.1145/3460120.3484560
- [5] Gilad Asharov, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Ariel Nof, Benny Pinkas, Katsumi Takahashi, and Junichi Tomida. 2022. Efficient Secure Three-Party Sorting with Applications to Data Analysis and Heavy Hitters. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (Los Angeles, CA, USA) (CCS '22). Association for Computing Machinery, New York, NY, USA, 125–138. doi:10.1145/3548606.3560691
- [6] Gilad Asharov, Koki Hamada, Ryo Kikuchi, Ariel Nof, Benny Pinkas, and Junichi Tomida. 2023. Secure Statistical Analysis on Multiple Datasets: Join and Group-By. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (Copenhagen, Denmark) (CCS '23). Association for Computing Machinery, New York, NY, USA, 3298–3312. doi:10.1145/3576915.3623119
- [7] Axel Bacher, Olivier Bodini, Alexandros Hollender, and Jérémie Lumbroso. 2015. Mergeshuffle: a very fast, parallel random permutation algorithm. *arXiv preprint 1508.03167* (2015).
- [8] Saikrishna Badrinarayanan, Sourav Das, Gayathri Garimella, Srinivasan Raghuraman, and Peter Rindal. 2022. Secret-Shared Joins with Multiplicity from Aggregation Trees. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (Los Angeles, CA, USA) (CCS '22). Association for Computing Machinery, New York, NY, USA, 209–222. doi:10.1145/3548606.3560670
- [9] Kenneth E. Batchier. 1968. Sorting Networks and Their Applications. In *American Federation of Information Processing Societies: AFIPS Conference Proceedings: 1968 Spring Joint Computer Conference, Atlantic City, NJ, USA, 30 April - 2 May 1968*, 307–314. doi:10.1145/1468075.1468121
- [10] Johes Bater, Gregory Elliott, Craig Eggen, Satyender Goel, Abel N. Kho, and Jennie Rogers. 2017. SMCQL: Secure Query Processing for Private Data Networks. *Proc. VLDB Endow.* 10, 6 (2017), 673–684. doi:10.14778/3055330.3055334
- [11] Johes Bater, Xi He, William Ehrich, Ashwin Machanavajjhala, and Jennie Rogers. 2018. Shrinkwrap: efficient sql query processing in

- differentially private data federations. *Proceedings of the VLDB Endowment* 12, 3 (2018), 307–320.
- [12] Johes Bater, Yongjoo Park, Xi He, Xiao Wang, and Jennie Rogers. 2020. SAQE: practical privacy-preserving approximate query processing for data federations. *Proceedings of the VLDB Endowment* 13, 12 (2020), 2691–2705.
 - [13] Laura Blackstone, Seny Kamara, and Tarik Moataz. 2020. Revisiting Leakage Abuse Attacks. In *NDSS*. The Internet Society.
 - [14] Marina Blanton and Everaldo Aguiar. 2012. Private and oblivious set and multiset operations. In *AsiaCCS*. ACM, 40–41.
 - [15] Dan Bogdanov, Liina Kamm, Baldur Kubo, Reimo Rebane, Ville Sokk, and Riivo Talviste. 2016. Students and Taxes: a Privacy-Preserving Study Using Secure Computation. *Proceedings on Privacy Enhancing Technologies (PoPETS)* 2016, 3 (2016), 117–135. <http://www.degruyter.com/view/j/popets.2016.2016.issue-3/popets-2015-0019/popets-2016-0019.xml>
 - [16] Dan Bogdanov, Sven Laur, and Riivo Talviste. 2014. A Practical Analysis of Oblivious Sorting Algorithms for Secure Multi-party Computation. In *Secure IT Systems*, Karin Bernsmed and Simone Fischer-Hübner (Eds.). Springer International Publishing, Cham, 59–74.
 - [17] Boston Women's Workforce Council. 2024. Data Privacy: Ensuring Secure and Private Data Analysis. <https://thebwcc.org/mpc>.
 - [18] Henry Corrigan-Gibbs and Dan Boneh. 2017. Prio: Private, Robust, and Scalable Computation of Aggregate Statistics. In *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, Boston, Massachusetts, USA, 259–282. <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/corrigan-gibbs>
 - [19] Anders P. K. Dalskov, Daniel Escudero, and Marcel Keller. 2021. Fantastic Four: Honest-Majority Four-Party Secure Computation With Malicious Security. In *USENIX Security Symposium*. USENIX Association, 2183–2200.
 - [20] Ivan Damgård and Jesper Buus Nielsen. 2003. Universally Composable Efficient Multiparty Computation from Threshold Homomorphic Encryption. In *CRYPTO (Lecture Notes in Computer Science, Vol. 2729)*. Springer, 247–264.
 - [21] Emma Dauterman, Mayank Rathee, Raluca Ada Popa, and Ion Stoica. 2022. Waldo: A Private Time-Series Database from Function Secret Sharing. In *2022 IEEE Symposium on Security and Privacy (SP)*. 2450–2468. doi:10.1109/SP46214.2022.9833611
 - [22] Ankur Dave, Chester Leung, Raluca Ada Popa, Joseph E. Gonzalez, and Ion Stoica. 2020. Oblivious competitive analytics using hardware enclaves. In *EuroSys*. ACM, 39:1–39:17.
 - [23] Daniel Demmler, Thomas Schneider, and Michael Zohner. 2015. ABY - A Framework for Efficient Mixed-Protocol Secure Two-Party Computation. In *22nd Annual Network and Distributed System Security Symposium, NDSS 2015, San Diego, California, USA, February 8-11, 2015*. The Internet Society. <https://www.ndss-symposium.org/ndss2015/aby---framework-efficient-mixed-protocol-secure-two-party-computation>
 - [24] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, et al. 2019. The design and operation of {CloudLab}. In *2019 USENIX annual technical conference (USENIX ATC 19)*. 1–14.
 - [25] Daniel Escudero, Satrajit Ghosh, Marcel Keller, Rahul Rachuri, and Peter Scholl. 2020. Improved Primitives for MPC over Mixed Arithmetic-Binary Circuits. In *CRYPTO (2) (Lecture Notes in Computer Science, Vol. 12171)*. Springer, 823–852.
 - [26] Muhammad Faisal, Jerry Zhang, John Liagouris, Vasiliki Kalavri, and Mayank Varia. 2023. TVA: A multi-party computation system for secure and expressive time series analytics. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 5395–5412. <https://www.usenix.org/conference/usenixsecurity23/presentation/faisal>
 - [27] Wenjing Fang, Shunde Cao, Guojin Hua, Junming Ma, Yongqiang Yu, Qunshan Huang, Jun Feng, Jin Tan, Xiaopeng Zan, Pu Duan, et al. 2024. SecretFlow-SCQL: A Secure Collaborative Query Platform. *Proceedings of the VLDB Endowment* 17, 12 (2024), 3987–4000.
 - [28] Ronald Aylmer Fisher and Frank Yates. 1938. *Statistical Tables for Biological, Agricultural and Medical Research*. Oliver and Boyd.
 - [29] Michael T. Goodrich. 2011. Data-oblivious external-memory algorithms for the compaction, selection, and sorting of outsourced data. In *SPAA*. ACM, 379–388.
 - [30] J. Gray, A. Bosworth, A. Lyaman, and H. Pirahesh. 1996. Data cube: a relational aggregation operator generalizing GROUP-BY, CROSS-TAB, and SUB-TOTALS. In *Proceedings of the Twelfth International Conference on Data Engineering*. 152–159. doi:10.1109/ICDE.1996.492099
 - [31] Paul Grubbs, Marie-Sarah Lacharité, Brice Minaud, and Kenneth G. Paterson. 2018. Pump up the Volume: Practical Database Reconstruction from Volume Leakage on Range Queries. In *CCS*. ACM, 315–331.
 - [32] Koki Hamada, Ryo Kikuchi, Dai Ikarashi, Koji Chida, and Katsumi Takahashi. 2012. Practically Efficient Multi-party Sorting Protocols from Comparison Sort Algorithms. In *ICISC (Lecture Notes in Computer Science, Vol. 7839)*. Springer, 202–216.
 - [33] Feng Han, Lan Zhang, Hanwen Feng, Weiran Liu, and Xiangyang Li. 2022. Scape: Scalable Collaborative Analytics System on Private Database with Malicious Security. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. 1740–1753. doi:10.1109/ICDE53745.2022.00176
 - [34] W Daniel Hillis and Guy L Steele Jr. 1986. Data parallel algorithms. *Commun. ACM* 29, 12 (1986), 1170–1183.
 - [35] Paulo Jesus, Carlos Baquero, and Paulo Sérgio Almeida. 2014. A survey of distributed data aggregation algorithms. *IEEE Communications Surveys & Tutorials* 17, 1 (2014), 381–404.
 - [36] Manas R. Joglekar, Rohan Puttagunta, and Christopher Ré. 2016. AJAR: Aggregations and Joins over Annotated Relations. In *Proceedings of the 35th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, PODS 2016, San Francisco, CA, USA, June 26 - July 01, 2016*, Tova Milo and Wang-Chiew Tan (Eds.). ACM, 91–106. doi:10.1145/2902251.2902293
 - [37] Kristján Valur Jónsson, Gunnar Kreitz, and Misbah Uddin. 2011. Secure multi-party sorting and applications. In *Proceedings of the 9th International Conference on Applied Cryptography and Network Security (ACNS) (Nerja (Malaga), Spain)*.
 - [38] Seny Kamara, Abdelkarim Kati, Tarik Moataz, Jamie DeMaria, Andrew Park, and Amos Treiber. 2024. MAPLE: MArkov Process Leakage attacks on Encrypted Search. *Proc. Priv. Enhancing Technol.* 2024, 1 (2024), 430–446.
 - [39] Seny Kamara, Abdelkarim Kati, Tarik Moataz, Thomas Schneider, Amos Treiber, and Michael Yonli. 2022. SoK: Cryptanalysis of Encrypted Search with LEAKER - A framework for LEAKage Attack Evaluation on Real-world data. In *EuroS&P*. IEEE, 90–108.
 - [40] Darya Kaviani, Sijun Tan, Pravein Govindan Kannan, and Raluca Ada Poda. 2024. Flock: A Framework for Deploying On-Demand Distributed Trust. In *USENIX Symposium on Operating Systems Design and Implementation*.
 - [41] Kate Keahey, Jason Anderson, Zhuo Zhen, Pierre Riteau, Paul Ruth, Dan Stanzione, Mert Cevik, Jacob Collieran, Haryadi S. Gunawi, Cody Hammock, Joe Mambretti, Alexander Barnes, François Halbach, Alex Rocha, and Joe Stubbs. 2020. Lessons Learned from the Chameleon Testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association.
 - [42] Marcel Keller. 2020. MP-SPDZ: A versatile framework for multi-party computation. In *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*. 1575–1590.
 - [43] B. Knott, S. Venkataraman, A.Y. Hannun, S. Sengupta, M. Ibrahim, and L.J.P. van der Maaten. 2020. CrypTen: Secure Multi-Party Computation Meets Machine Learning. In *Proceedings of the NeurIPS Workshop on*

Privacy-Preserving Machine Learning.

- [44] Simeon Krastnikov, Florian Kerschbaum, and Douglas Stebila. 2020. Efficient Oblivious Database Joins. *Proc. VLDB Endow.* 13, 11 (2020), 2132–2145. <http://www.vldb.org/pvldb/vol13/p2132-krastnikov.pdf>
- [45] John Liagouris, Vasiliki Kalavri, Muhammad Faisal, and Mayank Varia. 2023. SECRECY: Secure collaborative analytics in untrusted clouds. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 1031–1056. <https://www.usenix.org/conference/nsdi23/presentation/liagouris>
- [46] The libsodium Community. 2025. libsodium: A modern, portable, easy to use crypto library. <https://libsodium.org/>. [Online; accessed September 2025].
- [47] Yehuda Lindell. 2020. Secure multiparty computation. *Commun. ACM* 64, 1 (dec 2020), 86–96. doi:10.1145/3387108
- [48] Fukang Liu, Takanori Isobe, and Willi Meier. 2021. Cryptanalysis of Full LowMC and LowMC-M with Algebraic Techniques. In *Advances in Cryptology – CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part III*. Springer-Verlag, Berlin, Heidelberg, 368–401. doi:10.1007/978-3-030-84252-9_13
- [49] Mi Lu. 2005. *Arithmetic and logic in computer systems*. John Wiley & Sons.
- [50] Qiyao Luo, Yilei Wang, Wei Dong, and Ke Yi. 2024. Secure Query Processing with Linear Complexity. *arXiv preprint 2403.13492* (2024). arXiv:2403.13492 [cs.CR] <https://arxiv.org/abs/2403.13492>
- [51] Junming Ma, Yancheng Zheng, Jun Feng, Derun Zhao, Haoqi Wu, Wenjing Fang, Jin Tan, Chaofan Yu, Benyu Zhang, and Lei Wang. 2023. SecretFlow-SPU: A Performant and User-Friendly Framework for Privacy-Preserving Machine Learning. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 17–33. <https://www.usenix.org/conference/atc23/presentation/ma>
- [52] Samuel R. Madden, Michael J. Franklin, Joseph M. Hellerstein, and Wei Hong. 2005. TinyDB: an acquisitional query processing system for sensor networks. *ACM Trans. Database Syst.* 30, 1 (March 2005), 122–173. doi:10.1145/1061318.1061322
- [53] Apostolos Mavrogiannakis, Xian Wang, Ioannis Demertzis, Dimitrios Papadopoulos, and Minos Garofalakis. 2025. OBLIVIATOR: Oblivious Parallel Joins and other Operators in Shared Memory Environments. *Cryptology ePrint Archive*, Paper 2025/183. <https://eprint.iacr.org/2025/183>
- [54] C.J.H. McDiarmid and R.B. Hayward. 1996. Large Deviations for Quicksort. *J. Algorithms* 21, 3 (Nov. 1996), 476–507. doi:10.1006/jagm.1996.0055
- [55] Payman Mohassel, Peter Rindal, and Mike Rosulek. 2020. Fast Database Joins and PSI for Secret Shared Data. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (Virtual Event, USA) (CCS '20)*. Association for Computing Machinery, New York, NY, USA, 1271–1287. doi:10.1145/3372297.3423358
- [56] Mozilla. 2019. Next steps in privacy-preserving telemetry with Prio. <https://blog.mozilla.org/security/2019/06/06/next-steps-in-privacy-preserving-telemetry-with-prio/>. [Online; accessed September 2025].
- [57] Muhammad Naveed, Seny Kamara, and Charles V. Wright. 2015. Inference Attacks on Property-Preserving Encrypted Databases. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS '15)*. 644–655. doi:10.1145/2810103.2813651
- [58] Thomas Neumann and Viktor Leis. 2024. A Critique of Modern SQL and a Proposal Towards a Simple and Expressive Query Language. In *14th Conference on Innovative Data Systems Research, CIDR 2024, Chaminade, HI, USA, January 14–17, 2024*. www.cidrdb.org. <https://www.cidrdb.org/cidr2024/papers/p48-neumann.pdf>
- [59] Christopher Olston, Benjamin Reed, Adam Silberstein, and Utkarsh Srivastava. 2008. Automatic optimization of parallel dataflow programs. In *USENIX 2008 Annual Technical Conference (Boston, Massachusetts) (ATC'08)*. USENIX Association, USA, 267–273.
- [60] Stanislav Peceny, Srinivasan Raghuraman, Peter Rindal, and Harshal Shah. 2024. Efficient Permutation Correlations and Batched Random Access for Two-Party Computation. *Cryptology ePrint Archive*, Paper 2024/547. <https://eprint.iacr.org/2024/547>
- [61] Xinyu Peng, Feng Han, Li Peng, Weiran Liu, Zheng Yan, Kai Kang, Xinyuan Zhang, Guoxing Wei, Jianling Sun, and Jinfei Liu. 2024. MapComp: A Secure View-based Collaborative Analytics Framework for Join-Group-Aggregation. *arXiv preprint 2408.01246* (2024). arXiv:2408.01246 [cs.CR] <https://arxiv.org/abs/2408.01246>
- [62] Rishabh Poddar, Sukrit Kalra, Avishay Yanai, Ryan Deng, Raluca Ada Popa, and Joseph M Hellerstein. 2021. Senate: A Maliciously-Secure MPC Platform for Collaborative Analytics. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Vancouver, B.C. <https://www.usenix.org/conference/usenixsecurity21/presentation/poddar>
- [63] Mayank Rathee, Yuwen Zhang, Henry Corrigan-Gibbs, and Raluca Ada Popa. 2024. Private Analytics via Streaming, Sketching, and Silently Verifiable Proofs. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 194–194.
- [64] Peter Rindal and Lance Roy. Last access: September 2024. libOTe: an efficient, portable, and easy to use Oblivious Transfer Library. <https://github.com/osu-crypto/libOTe>.
- [65] Adi Shamir. 1979. How to Share a Secret. *Commun. ACM* 22, 11 (Nov. 1979), 612–613. doi:10.1145/359168.359176
- [66] SQLite. 2025. SQLite SQL database engine. <https://sqlite.org/>. [Online; accessed September 2025].
- [67] Transaction Processing Performance Council. 2024. TPC-H Benchmark Specification. <https://www.tpc.org/tpch/>. [Online; accessed September 2024].
- [68] Transaction Processing Performance Council. 2024. TPC-H Benchmark Specification (Query Definitions). https://www.tpc.org/TPC_Documents_Current_Versions/pdf/TPC-H_v3.0.1.pdf. [Online; accessed September 2024].
- [69] United Nations Global Working Group (GWG) Task Team on Privacy Preserving Techniques. 2023. Case study repository. <https://unstats.un.org/wiki/display/UGTTOPPT/Case+study+repository>.
- [70] Nikolaj Volgushev, Malte Schwarzkopf, Ben Getchell, Mayank Varia, Andrei Lapets, and Azer Bestavros. 2019. Conclave: secure multi-party computation on big data. In *Proceedings of the Fourteenth EuroSys Conference 2019, Dresden, Germany, March 25–28, 2019*, George Candea, Robbert van Renesse, and Christof Fetzer (Eds.). ACM, 3:1–3:18. doi:10.1145/3302424.3303982
- [71] Yilei Wang and Ke Yi. 2021. *Secure Yannakakis: Join-Aggregate Queries over Private Data*. Association for Computing Machinery, New York, NY, USA, 1969–1981. <https://doi.org/10.1145/3448016.3452808>
- [72] Mihalis Yannakakis. 1981. Algorithms for acyclic database schemes. In *VLDB*, Vol. 81. 82–94.
- [73] Yuan Yu, Pradeep Kumar Gunda, and Michael Isard. 2009. Distributed aggregation for data-parallel computing: interfaces and implementations. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (Big Sky, Montana, USA) (SOSP '09)*. Association for Computing Machinery, New York, NY, USA, 247–260. doi:10.1145/1629575.1629600
- [74] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivararam Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. 2016. Apache Spark: a unified engine for big data processing. *Commun. ACM* 59, 11 (2016), 56–65. doi:10.1145/2934664
- [75] Wenhao Zhang, Xiaojie Guo, Kang Yang, Ruiyu Zhu, Yu Yu, and Xiao Wang. 2024. Efficient Actively Secure DPF and RAM-based 2PC with One-Bit Leakage. In *IEEE Symposium on Security and Privacy, SP 2024*,

San Francisco, CA, USA, May 19-23, 2024. IEEE, 561–577. doi:10.1109/SP54263.2024.00205

- [76] Wenting Zheng, Ankur Dave, Jethro G. Beekman, Raluca Ada Popa, Joseph E. Gonzalez, and Ion Stoica. 2017. Opaque: An Oblivious and Encrypted Distributed Analytics Platform. In *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. Boston, Massachusetts, USA, 283–298. <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/zheng>

Appendix

In the following sections, we provide additional details on our oblivious shuffling primitives (Appendix A), sorting (including quicksort, radixsort, and our table sort protocol; Appendix B), and join (including arguments of correctness and more information on the trimming heuristic; Appendix C). Appendix D provides detailed bandwidth measurements for ORQ and the systems we compare against, and Appendix E shows ORQ's performance in a geodistributed setting. Appendices are not included in the peer-review process.

A Oblivious Shuffle

This section describes a set of primitives related to obliviously shuffling a secret-shared vector. Existing works typically discuss such primitives for a fixed number of parties and specific MPC protocol (e.g., [5, 60]). Our primary contribution in this section is a single stack of primitives that can be used across a variety of MPC protocols and threat models. We also provide novel algorithms for obliviously inverting elementwise permutations and converting elementwise permutations between arithmetic and boolean sharings.

A.1 Preliminaries

A permutation is a bijective mapping from $[n]$ to $[n]$, where $[n]$ represents the set $\{1, \dots, n\}$. We denote the composition of permutations π and ρ by $\pi \circ \rho(\cdot) = \pi(\rho(\cdot))$. We represent permutations as index maps: starting from a data array $\vec{x}$, if $\pi_i = j$ then $\pi(\vec{x})$ maps the value x_i to position j . As a special case, a *sorting permutation* for an input vector $\vec{x}$ is a permutation σ such that $\sigma(\vec{x})$ is sorted. We use $\mathbb{I}$ to denote the identity permutation $\mathbb{I} = (1, \dots, n)$. We use $\llbracket x \rrbracket$ to denote that a value x is secret-shared, and we do not distinguish in notation between an arithmetic and boolean sharing.

Like Asharov et al. [5], in this work we consider two different forms of secret-sharing for permutations:

Elementwise Permutation. A vector of secret-shared indices, denoted $\llbracket \pi \rrbracket = (\llbracket \pi_1 \rrbracket, \dots, \llbracket \pi_n \rrbracket)$. The elements can either be arithmetic or boolean secret-sharings.

Sharded Permutation. A composition of random permutations, denoted $\langle \pi \rangle = \pi_n \circ \dots \circ \pi_1$.

We reproduce an important fact about permutations from Observation 2.4 of Asharov et al. [5].

Fact 1. Let π and σ be permutations over the set $[m]$ and let $\vec{\sigma} = (\sigma(1), \dots, \sigma(m))$ be the vector of destinations of the permutation σ . Then $\pi(\vec{\sigma}) = \sigma \circ \pi^{-1}([m])$, i.e. a vector of destinations for the permutation $\sigma \circ \pi^{-1}$.

A.2 Local permutations

Many of the oblivious shuffling primitives that will be discussed in the following sections involve locally generating and locally applying random permutations as subroutines. In this section, we discuss the algorithms used for generating

and applying random local permutations and our methods for parallelizing these operations.

Generating random local permutations. To generate local random permutations, we use the Fisher-Yates shuffle algorithm [28], a single-threaded algorithm. While there exist algorithms for generating random permutations in parallel (such as MergeShuffle [7]), we typically need to generate many random permutations at once. As a result, we parallelize the generation by distributing a batch of permutations among all cores, so that each permutation is generated in a single-threaded fashion.

We may additionally wish to generate identical random permutations among multiple parties. This can be achieved by having each party generate the permutation locally using a pseudorandom generator (PRG) with the same seed.

Applying random local permutations. Unlike generation, we apply random local permutations one at a time, so we need to parallelize each individual local permutation application. To apply a local permutation, we want each element x_i of a vector $\vec{x}$ to be placed at index π_i in the output vector $\vec{y}$. We can easily divide $\vec{x}$ and π into contiguous blocks and give one to each thread. Since we must access random elements in $\vec{y}$, we cannot give each thread a contiguous block of $\vec{y}$. However, we can give each thread full write access to $\vec{y}$ and mathematically guarantee that no element will be written to more than once.

A.3 Sharded permutation protocols

In this section, we describe protocols to generate and apply sharded permutations, which we denote as `genShardedPerm` and `applyShardedPerm`, respectively. These permutations apply to all three MPC protocols used in ORQ. We explain separately our protocols in the honest-majority and dishonest-majority settings.

Honest majority. In the three-party setting with replicated secret-sharing, we use the methods for generating and applying sharded permutations by Asharov et al. [5], which we restate below for completeness. A sharded permutation $\langle \pi \rangle = \pi_3 \circ \pi_2 \circ \pi_1$ can be expressed as a replicated secret sharing

$$\langle \pi \rangle = ((\pi_1, \pi_2), (\pi_2, \pi_3), (\pi_3, \pi_1)).$$

That is, each party P_i holds two permutations: one in common with P_{i+1} and one in common with P_{i-1} . To generate such a sharing, each pair of parties generates a random local permutation using their common PRG seed. However, there is one permutation that P_i does not know, so π remains secret.

Later when the parties call `applyShardedPerm` to apply a sharded permutation, each pair of parties locally applies their common permutation and reshapes the permuted result to the excluded party. First, P_1 and P_3 permute under π_1 and reshare to P_2 , and then so on for π_2 and π_3 .

We generalize the protocol to work with any replicated secret-sharing scheme in the honest-majority setting, with an emphasis on the four-party Fantastic Four [19] protocol. The three-party protocol proceeds in a sequence of rounds, where in each round a subset of the parties—which we will call a *shuffle group*—locally permutes the vector and reshapes it to the remaining party. We generalize this idea of shuffle groups and design protocols in which each round contains a local permutation application and resharing by a single shuffle group. For security, we require that shuffle groups obey two properties:

1. Each shuffle group can collectively hold a sharing of the input data. As a result, the size of a shuffle group is greater than T , which is why this approach is limited to the honest-majority setting.
2. There must exist at least one shuffle group containing no corrupted parties, so that the composed permutation π is unknown to the adversary.

Using the Fantastic Four protocol as a concrete example [19], here is a set of shuffle groups in the semi-honest setting: $\mathcal{G} = \{\{P_0, P_1\}, \{P_2, P_3\}\}$. Since there is only $T = 1$ corrupted party, each shuffle group possesses at least one copy of all secret shares, and there is a shuffle group with no corrupted parties.

There are two ways to extend this idea to the malicious-secure four-party setting. One trivial approach is to combine the above idea with the generic compiler from semi-honest to malicious security for the reshare functionality proposed by Asharov et al. [5]. However, we take a different approach that provides ease of implementation and consistency with the other functionalities in the Fantastic Four protocol [19]. We use four shuffle groups of three parties each, so that each share is contained twice in each group, allowing for redundancy in resharing the value to the excluded party:

$$\mathcal{G} = \{\{P_0, P_1, P_2\}, \{P_1, P_2, P_3\}, \{P_2, P_3, P_0\}, \{P_3, P_0, P_1\}\}.$$

We can then base the malicious security of the oblivious sharded permutation application on the black-box security of the INP protocol described in Fantastic Four [19]. Specifically, the receiving party receives either two copies of each share or one copy and its hash. In either case, a single corrupted party cannot corrupt both values received, so cheating will be detected with overwhelming probability.

Dishonest majority. In the two-party setting with one dishonest party, we cannot use shuffle groups and therefore instead adopt the framework of Pecený et al. [60] for generating and applying sharded permutations. Their approach is based on a *permutation correlation*, which is a pair of tuples, one held by each party: $(A, B_0), (B_1, \pi)$ such that $\pi(A) = B_0 + B_1$. Here, $A, B_0, B_1 \in \mathbb{F}^n$ and π is a random permutation over n elements. B_0 and B_1 can be either arithmetic or boolean secret-shares, with the addition operator defined correspondingly.

To generate a sharded permutation, it suffices to generate two permutation correlations in a preprocessing phase, one with each party as the sender. Pecený et al. [60] propose two protocols for generating permutation correlations, both based on oblivious pseudorandom functions (OPRFs), with one of them additionally using pseudorandom correlation generators to achieve communication sublinear in the length of the elements (as opposed to the length of the permutations).

To apply a sharded permutation obliviously, we use the protocol $\Pi_{\text{Comp-Perm}}$ from Pecený et al. [60]. We make use of their library’s OPRF implementation [1, 60] and construct the rest of the permutation correlations from scratch. Similar ideas can be used to build `applyInverseShardedPerm` that applies the inverse of a sharded permutation. Since the output of the OPRF is a boolean secret sharing, we generate all permutation correlations with a boolean sharing and convert to an arithmetic sharing where needed using a boolean-to-arithmetic conversion protocol.

A.4 Shuffling framework

Our framework provides generic interfaces to the functionalities `genShardedPerm`, `applyShardedPerm`, and `applyInverseShardedPerm` that call the protocols to generate and apply (respectively) a sharded permutation as described in the previous section depending on the desired MPC protocol and threat model. In this section, we describe our abstract interface and the protocols for oblivious shuffling primitives. A detailed analysis of the complexity of each protocol in each setting can be found in Table 1.

The Permutation Manager abstraction. We now describe the `PermutationManager`, our abstraction for generating sharded permutations in a setting-agnostic manner. The goal is to make all differences in the generation and application of sharded permutations invisible to the higher level shuffle protocols. The `PermutationManager` exposes two functions:

- `genShardedPerm<T>(size, enc)`
- `genShardedPermPair<T1,T2>(size, enc1, enc2)`

The function `genShardedPerm` returns a random sharded permutation over `size` elements of type `T` (e.g., `int32`), where the elements have encoding `enc` (either an arithmetic or boolean sharing). The function `genShardedPermPair` returns two random sharded permutations over `size` elements such that they correspond to the same random permutation. This is necessary if we wish to permute multiple columns of data according to the same sharded permutation, as is the case, for example, when applying or composing elementwise permutations. The two vectors have types `T1` and `T2` and encodings `enc1` and `enc2`. Both functions allow for batched, parallel generation of many sharded permutations. Since all computation is data-independent, our framework generates sufficiently-many sharded permutations in preprocessing;

when called, these functions fetch an already-generated permutation.

In the honest-majority setting, sharded permutations are identical for all input types, regardless of the bitwidth or sharing type of the vectors to be permuted. To reserve a pair of sharded permutations which represent the same permutation, we simply generate a single sharded permutation $\langle \pi \rangle$ and return the pair $(\langle \pi \rangle, \langle \pi \rangle)$. The need to distinguish between a single sharded permutation and a pair of sharded permutations only arises in the dishonest-majority setting, as the underlying permutation correlations are type-dependent and cannot be securely reused.

Applying and composing permutations. As described in Section A.3, our framework provides methods that apply a sharded permutation and its inverse. It also provides methods for oblivious shuffling, permutation composition, and applying an elementwise permutation.

Our oblivious shuffle protocol `shuffle` simply generates and applies a sharded permutation, as shown in Protocol 4.

Protocol 4: Π_{Shuffle} : shuffle

input : $[[x]]$
output : $[[\pi(x)]]$ for random π

- 1 $\langle \pi \rangle \leftarrow \text{genShardedPerm}(\text{type}(x), \text{size}(x), \text{enc}(x))$
- 2 $[[\pi(x)]] \leftarrow \text{applyShardedPerm}([x], \langle \pi \rangle)$
- 3 **return** $[[\pi(x)]]$

Next, Protocol 5 shows `applyElementwisePerm`, which generalizes protocol 4.2 of Asharov et al. [5]. The main idea here (which we use also in subsequent protocols) is that if we obviously shuffle the elementwise permutation, then it is safe to open the shuffled vector in line 6 while revealing nothing about the unshuffled vector.

Protocol 5: $\Pi_{\text{ApplyElem}}$: applyElementwisePerm

input : $[[x]], [[\rho]]$
output : $[[\rho(x)]]$

- 1 $\tau_1, \tau_2 \leftarrow \text{type}(x), \text{type}(\rho)$
- 2 $\epsilon_1, \epsilon_2 \leftarrow \text{enc}(x), \text{enc}(\rho)$
- 3 $\langle \pi_1 \rangle, \langle \pi_2 \rangle \leftarrow \text{genShardedPermPair}(\tau_1, \tau_2, \text{size}(x), \epsilon_1, \epsilon_2)$
- 4 $[[\pi(x)]] \leftarrow \text{applyShardedPerm}([x], \langle \pi_1 \rangle)$
- 5 $[[\pi(\rho)]] \leftarrow \text{applyShardedPerm}([\rho], \langle \pi_2 \rangle)$
- 6 $\pi(\rho) \leftarrow \text{open}([[\pi(\rho)]])$
- 7 $[[\rho(x)]] \leftarrow \text{localApplyPerm}([[\pi(x)]], \pi(\rho))$
- 8 **return** $[[\rho(x)]]$

Finally, Protocol 6 shows `composePerms`, our method to compose two permutations $[[\sigma]]$ and $[[\rho]]$ that is based on protocol 4.3 of Asharov et al. [5]. For simplicity, we describe

the case where both permutations have the same encoding (either arithmetic or boolean). Otherwise, we can convert one encoding using the conversion protocol described next. Alternatively, we could allow the input elementwise permutations to have different sharing types, but we would then have to pay the additional cost of generating a pair of sharded permutations rather than a single sharded permutation.

Protocol 6: Π_{Compose} : composePerms

input : $[[\sigma]], [[\rho]]$
output : $[[\rho \circ \sigma]]$

- 1 $\langle \pi \rangle \leftarrow \text{genShardedPerm}(\text{type}(\sigma), \text{size}(\sigma), \text{enc}(\sigma))$
- 2 $[[\pi(\sigma)]] \leftarrow \text{applyShardedPerm}([[\sigma]], \langle \pi \rangle)$
- 3 $\pi(\sigma) \leftarrow \text{open}([[\pi(\sigma)]])$
- 4 $[[\pi \circ \sigma^{-1}(\rho)]] \leftarrow \text{localApplyPerm}([[\rho]], (\pi(\sigma))^{-1})$
- 5 $[[\rho \circ \sigma]] \leftarrow \text{applyInverseShardedPerm}([[\pi \circ \sigma^{-1}(\rho)]], \langle \pi \rangle)$
- 6 **return** $[[\rho \circ \sigma]]$

Novel shuffle primitives. We propose two novel protocols related to oblivious shuffling. The first converts an elementwise permutation from either an arithmetic sharing to a boolean sharing or from a boolean sharing to an arithmetic sharing. The second inverts an elementwise permutation.

To convert between arithmetic and boolean encodings, one trivial approach is to use arithmetic to boolean share conversions for each element of the permutation. In the dishonest-majority setting, we use this trivial approach. However, in the honest-majority setting, we can take advantage of the additional structure imposed by permutations by recognizing that we already know every value contained in the plaintext vector, but not the ordering. Specifically, we can open a shuffled version of the vector, re-share the opened vector under the desired type, and obviously apply the inverse of the shuffle. This approach, shown in Protocol 7, is faster than the trivial protocol in the honest-majority.

Protocol 7: Π_{Conv} : convertElementwisePerm

input : $[[x]]_T, T \in \{A : 0, B : 1\}$
output : $[[x]]_{1-T}$

- 1 $\tau \leftarrow \text{type}(x)$
- 2 $\langle \pi_1 \rangle, \langle \pi_2 \rangle \leftarrow \text{genShardedPerm}(\tau, \tau, (x, T, 1 - T))$
- 3 $[[\pi(x)]]_T \leftarrow \text{applyShardedPerm}([x]_T, \langle \pi_1 \rangle)$
- 4 $\pi(x) \leftarrow \text{reveal}([[\pi(x)]]_T)$
- 5 $[[\pi(x)]]_{1-T} \leftarrow \text{secretShare}(\pi(x), 1 - T)$
- 6 $[[x]]_{1-T} \leftarrow \text{applyInverseShardedPerm}([[\pi(x)]]_{1-T}, \langle \pi_2 \rangle)$
- 7 **return** $[[x]]_{1-T}$

We now describe a protocol for inverting an elementwise permutation. Whereas in `applyInverseShardedPerm` the

Primitive	2PC		3PC		4PC	
	Comm.	Rounds	Comm.	Rounds	Comm.	Rounds
genSharded	preprocessing	preprocessing	-	-	-	-
applySharded	$2\ell n$	2	$6\ell n$	3	$24\ell n$	4
shuffle	$2\ell n$	2	$6\ell n$	3	$24\ell n$	4
applyElementwise	$2\ell n + 3\ell_\sigma n$	5	$6\ell n + 7\ell_\sigma n$	7	$24\ell n + 25\ell_\sigma n$	9
compose	$5\ell_\sigma n$	5	$13\ell_\sigma n$	7	$49\ell_\sigma n$	9
invertElementwise	$5\ell_\sigma n$	5	$13\ell_\sigma n$	7	$49\ell_\sigma n$	9
convertElementwise	$5\ell_\sigma n$	5	$13\ell_\sigma n$	7	$49\ell_\sigma n$	9

Table 1. Communication and round complexities for 2PC, 3PC, and 4PC shuffling primitives. Here, n denotes the number of elements in the input vector, ℓ denotes the bitwidth of the elements, and ℓ_σ denotes the bitwidth of a permutation ($\ell_\sigma = 32$ in our system).

inverse was not computed directly but applied to a vector, here we wish to compute the inverse so it can be composed with other elementwise permutations. The protocol Π_{Inv} is simple: to invert an elementwise permutation, we simply obviously apply it to the identity permutation. Let $[[\pi]]$ be the permutation we wish to invert. We create an identity vector $\mathbb{I} = (1, \dots, n)$ and secret-share it. We then obviously apply $[[\pi]]$ to $[[\mathbb{I}]]$. The permuted identity vector is π^{-1} . By Fact 1, $\pi(\mathbb{I}) = \mathbb{I} \circ \pi^{-1} = \pi^{-1}$. We provide a formal description in Protocol 8, and we discuss how to use such an inversion protocol to extract the sorting permutation from a sorting protocol in section B.2.

Protocol 8: $\Pi_{\text{Inv}} : \text{invertElementwisePerm}$

input : $[[\pi]]$
output : $[[\pi^{-1}]]$
1 $[[\pi^{-1}]] \leftarrow \text{applyElementwisePerm}([[\mathbb{I}]], [[\pi]])$
2 **return** $[[\pi^{-1}]]$

B Oblivious Sorting

We implement two oblivious sorting protocols, quicksort and radixsort, each competitive with the state of the art in terms of performance. In §B.1, we describe the ‘base’ version of each protocol as an in-place, ascending-order sorting protocol that requires unique keys. In §B.2, we describe a wrapper protocol that removes these restrictions and allows us to extract the sorting permutation applied to the input as an elementwise permutation, which we then use to sort multiple columns in a table. In §B.3, we compare our radixsort protocol in detail with the prior state of the art. Finally, in §B.4, we discuss a probabilistic bound on the amount of preprocessing required for two-party quicksort.

Note on sorting complexity in MPC. The lower bound on the complexity of general-purpose comparison-based sorting algorithms for input size n is $\Omega(n \log n)$ comparisons. In MPC, comparisons are expensive operations, typically requiring $\Omega(\ell)$ communication, where ℓ is the bitwidth of the

elements. Hence, quicksort in MPC requires $\Omega(n \log n)$ comparison operations over ℓ -bit strings, for a total of $\Omega(\ell n \log n)$ bits of communication. Alternatively, sorting networks like bitonic sort involve $O(n \log^2 n)$ comparisons and require $\Omega(\ell n \log^2 n)$ communication [9]. Finally, radixsort [5] has either $O(\ell^2 n)$ or $O(\ell n \log n)$ communication depending on subtleties in the protocol that we discuss in §B.3.

In the general case where the plaintext vector might contain unique elements, our input space must have a bitwidth of at least $\ell = \Omega(\log n)$. Both quicksort and radixsort thus require $\Omega(n \log^2 n)$ communication, while bitonic sort requires $\Omega(n \log^3 n)$ communication. Hence, $\Omega(n \log^2 n)$ communication is the gold standard of general MPC sorting protocols.

B.1 Base sorting protocols

ORQ provides two base sorting protocols: an iterative version of quicksort and a radixsort protocol that combines features of Asharov et al. [5] and Bogdanov et al. [16]. The protocols only sort in ascending order, and quicksort requires unique keys in the sorting column; these restrictions simplify the protocols and allow for more effective optimization.

Quicksort. Quicksort is typically considered to be the most efficient general sorting algorithm in practice. However, it typically has a recursive and data-dependent control flow, both of which make a naïve application of quicksort to the MPC setting problematic in both efficiency and security.

To benefit from the efficiency of quicksort in the MPC setting, we use the *shuffle-then-sort* paradigm by Hamada et al. [32] in which we obviously shuffle the input vector first. As a result, the subsequent data-dependent sorting protocol only reveals the (meaningless) order of the shuffled vector. Additionally, we implement an iterative control flow for quicksort that executes all comparisons with a specified pivot in one round of comparisons. As a result, the sort takes $O(\log n)$ comparison rounds or $O(\log n \log \ell)$ communication rounds.

The resulting protocol is described in Protocol 9 below. All sorting happens in place, and the protocol alternates

between local computation and rounds of full vector comparisons. The protocol initializes a plaintext set of pivots S with the first element of the vector and iterates until S contains all elements in the vector. In each iteration, we compare every element at index i with one of the pivots: namely, the highest pivot index j such that $j \leq i$ via a function $j \leftarrow \text{prevPivot}(i)$ that can be computed locally by each party. We create a secret-shared vector $[[\vec{c}]]$ of all previous-pivot elements, perform all comparisons between $\vec{x}$ and $\vec{c}$ in parallel, and omit comparisons for pivot elements (which would be compared with themselves).

After each round of secure comparisons, we reveal the result vector $\vec{r}$ and then partition the vector based on the desired pivot locations. Concretely, for each pivot p , we define a partitioning subroutine $\text{partition}([[\vec{x}]], p, i, \vec{r})$ that takes as input the vector $[[\vec{x}]]$, the start index p where the pivot is located, an end index i , and $\vec{r}$. The subroutine moves items in the subvector according to standard quicksort: all items less than the pivot, then the pivot itself, and all items greater than the pivot. It returns a set S' of up to three locations for the pivot position and the next-round pivots for the left and right halves. This step is entirely local since S' and $\vec{r}$ are common knowledge.

To improve concrete efficiency, we can represent S as a scaled indicator vector, $S_i = i \cdot \mathbb{I}[i \text{ is a pivot}]$; i.e., if index i is a pivot, index i of S contains i ; otherwise, 0. Then, prevPivot is implementable via a single call to a prefix-max function (which computes a running maximum of a list). Likewise, the result of partition is no longer used to compute $S \cup S'$; instead, we update the new indices to contain their value, $\forall j \in S' : S_j = j$.

Protocol 9: Iterative Quicksort

```

input :  $[[\vec{x}]]$  containing unique elements
output :  $[[\vec{y}]] = \sigma([[\vec{x}]])$ 
1  $[[\vec{x}]] \leftarrow \text{shuffle}([[\vec{x}]])$ 
2  $S \leftarrow \{1\}$ 
3 while  $|S| \neq n$  do
4    $[[\vec{c}]] \leftarrow []$ 
5   for  $i$  from 1 to  $n$  do
6      $j \leftarrow \text{prevPivot}(i)$ 
7      $[[\vec{c}]]_i \leftarrow [[\vec{x}]]_j$ 
8    $[[\vec{r}]] \leftarrow \text{compare}([[\vec{x}]], [[\vec{c}]])$ 
9    $\vec{r} \leftarrow \text{open}([[\vec{r}]])$ 
10   $i \leftarrow n$ 
11  for  $p \in \text{reversed}(S)$  do
12     $S' \leftarrow \text{partition}([[\vec{x}]], p, i, \vec{r})$ 
13     $S \leftarrow S \cup S'$ 
14     $i \leftarrow p - 1$ 
15 return  $[[\vec{x}]]$ 

```

To see (informally) why this is secure, consider the first iteration of quicksort after shuffling a vector $\vec{x}$. Let the pivot be the first element, x_1 . We compare all elements $x_2, \dots, x_n$ to x_1 under MPC. Since x_1 is the first element *after shuffling*, it is drawn randomly from the original vector. The results of the comparisons are $n - 1$ bits revealing only x_1 's position in the sorted list but no information about the non-pivot elements: any $(n - 1)$ -bit string with the same Hamming weight is consistent with *multiple* permutations of the original vector. Since the shuffle permutation is drawn randomly, and is not known in plaintext to any single party, no information is revealed by opening the result of the comparisons to all parties.

We remark that Protocol 9 is only secure if all input elements in $[[\vec{x}]]$ are unique, so that each comparison check in $\vec{r}$ is equally likely to be true or false based only on the random shuffle and not the data $\vec{x}$. We remove this limitation in §B.2.

Radixsort. We now describe our radixsort protocol. We apply the recent advances in efficient oblivious shuffling to the radixsort protocol of Bogdanov et al. [16]. We find that for sorting key bitwidths that are reasonable in practice (e.g., $\ell = 32$ or 64), our protocol achieves lower round complexity and concretely better performance than the radixsort protocol of Asharov et al. [5] while being comparable in communication complexity, as shown in the performance analysis in §B.3.

Radixsort is shown in Protocol 10. The basic idea is simple: for each bit that we wish to sort by, we invoke the genBitPerm protocol of Asharov et al., which returns an elementwise sharing of the sorting permutation for that bit σ_i . Although genBitPerm was proposed in the honest-majority three-party setting, we observe that it is actually agnostic to the setting and the number of parties, and it only makes black-box use of basic MPC primitives. We then apply the permutation to the full vector by invoking the $\text{applyElementwisePerm}$ protocol.

It is sometimes desirable to sort only a subset of the bits, for instance when we want to prune a table and each element of the input vector $\vec{x}$ contains a binary flag bit indicating whether to keep this row. For this reason, the radixsort protocol takes two inputs beyond the data vector: the total number of bits to sort ℓ and the number of least significant bits to skip ℓ_s .

Protocol 10: Radixsort

```

input :  $[[\vec{x}]]$ ,  $\ell$ ,  $\ell_s$ 
output :  $[[\vec{y}]] = \sigma([[\vec{x}]])$ 
1  $[[\vec{y}]] \leftarrow [[\vec{x}]]$ 
2 for  $i$  from 1 to  $\ell$  do
3    $[[\sigma_i]] \leftarrow \text{genBitPerm}([[\vec{y}]] \gg (i + \ell_s))$ 
4    $[[\vec{y}]] \leftarrow \text{applyElementwisePerm}([[\vec{y}]], [[\sigma_i]])$ 
5 return  $[[\vec{y}]]$ 

```

B.2 Wrapper and table sort protocols

We now describe a general sorting wrapper protocol that takes as input a boolean secret-shared vector $[[\vec{x}]]$ to sort, the order in which to sort (either ascending (ASC) or descending (DESC)), and a base sorting protocol (either quicksort or radix-sort). As output, it returns the sorted vector $[[\vec{y}]] = \sigma([\vec{x}])$ and an elementwise sharing of the sorting permutation $[[\sigma]]$. The wrapper protocol has three steps: input padding, executing the base sorting protocol, and permutation extraction.

Input padding. Following Hamada et al. [32], for each element of the input vector $\vec{x}$, we append its index. This padding ensures that the elements are unique, that we can extract the sorting permutation, and that the sort is *stable* in the sense that if $x_i = x_j$ with $i < j$, then x_i will remain before x_j in the sorted vector. Concretely, to sort an input vector $[[\vec{x}]]$ in ascending order, we form the concatenated string $[[x'_i]] = [[x_i]] \parallel [[i]]$, with the sharing of i being a local operation using the publicShare protocol. To ensure that all n elements in $\vec{x}'$ are unique, we must add at least $\lceil \lg n \rceil$ bits of padding; our implementation always adds 32 bits of padding.

Permutation extraction. After the base sort of $\vec{x}'$ is completed, the i^{th} element in the input vector, with the padded value i , winds up at position σ_i . We then extract the sorting permutation from the padding bits so that we can apply it to other columns in a table; to the best of our knowledge, no prior works have detailed how to extract the sorting permutation from the padding. We separate the sorted list $\vec{y}'$ into the data $\vec{y} = \sigma(\vec{x})$ and a permutation π . The padding bits contain the result of applying the sorting permutation σ to the identity permutation, so $\pi = \sigma(\mathbb{I})$. By Fact 1, $\sigma(\mathbb{I}) = \mathbb{I} \circ \sigma^{-1} = \pi$, so $\sigma = \pi^{-1}$. Hence, we obtain the sorting permutation σ by inverting π using the invertElementwisePerm protocol.

Descending order. Sorting an input vector $\vec{x}$ in descending order can be done similarly. As a thought experiment: it almost works to use the ascending-sort protocol above and locally reverse the output vector, except it breaks stability by reversing the tiebreak condition. That is, if $x_i = x_j$ and $i < j$, then ascending-sort preserves stability but the local reverse ensures that x_i always comes after x_j instead.

To solve this issue, we negate the padding for each element. That is, each secret-shared element $[[x_i]]$ now gets padded to $[[x'_i]] = [[x_i]] \parallel [[-i]]$. Since we are attaching the padding bits as the least significant bits, we need any two values $-i$ and $-j$ where $i < j$ to satisfy $-i > -j$ even when interpreted as an unsigned integer. Our implementation one-indexes the input vector and uses two's complement encoding for the padding.

Putting it all together. We write the general sorting wrapper in Protocol 11 and summarize it here. We start by padding the input $[[\vec{x}]]$ with $\mathbb{I}_n$ in order to perform an ascending sort for input size n , or with $-1 \cdot \mathbb{I}_n$ to sort in descending order.

The resulting padded input vector $[[\vec{x}']]$ is fed into the base quicksort or radixsort protocol from §B.1, which results in the padded sorted vector $[[\vec{y}']]$. Finally, we locally split $[[\vec{y}']]$ into the unpadded result $[[\vec{y}]]$ and a permutation $[[\pi]]$ and invert $[[\pi]]$ to obtain the sorting permutation $[[\sigma]]$. If sorting in descending order, before inverting $[[\pi]]$, we convert it from boolean to arithmetic sharing to negate each element in the permutation, as constant multiplication is free for arithmetic shares. We return both $[[\vec{y}]]$ and $[[\sigma]]$.

Protocol 11: Sorting Wrapper Protocol

```

input :  $[[\vec{x}]]$ , order  $\in \{\text{ASC}, \text{DESC}\}$ ,
        sort  $\in \{\text{quicksort}, \text{radixsort}\}$ 
output :  $[[\vec{y}]] = \sigma([\vec{x}])$ ,  $[[\sigma]]$ 
1  $\vec{p} \leftarrow \mathbb{I}_n$ 
2 if order == DESC then
3    $\forall i \in [n], p_i \leftarrow -p_i$ 
4  $[[\vec{p}]] \leftarrow \text{publicShare}(\vec{p})$ 
5  $\forall i \in [n], [[x']]_i \leftarrow [[x]]_i \parallel [[p]]_i$ 
6  $[[\vec{y}']] \leftarrow \text{sort}([\vec{x}'])$ 
7  $[[\vec{y}]] \parallel [[\pi]] \leftarrow \text{split}([\vec{y}'])$ 
8 if order == DESC then
9    $[[\pi]] \leftarrow \text{convertElementwisePerm}([\pi], A)$ 
10  $\forall i \in [n], [[\pi]]_i \leftarrow -[[\pi]]_i$ 
11  $[[\sigma]] \leftarrow \text{invertElementwisePerm}([\pi])$ 
12 return  $[[\vec{y}]]$ ,  $[[\sigma]]$ 

```

Table sort. Finally, in Protocol 2 (in the main text) we use the wrapper protocol to design a simple and efficient protocol for sorting a table, where we may want to sort many columns of the table in a mix of ascending and descending order. Table sort takes as input an ordered list of columns to sort along with a corresponding direction and a list of additional columns to be permuted according to the sort columns. The protocol then sorts the sort columns from the least to the most significant column, and then applies the overall sorting permutation to all columns.

B.3 Radixsort analysis

Our radixsort is similar to the protocol of Bogdanov et al. [16], except we make use of recent advances in oblivious shuffling discussed in §A.4. For reasonable bitwidths like $\ell = 32$, our protocol can be seen as an optimization of Asharov et al. [5] that achieves a significant improvement in round complexity and a mild improvement in communication complexity.

Comparison. The key difference between our works is that, whereas Asharov et al. runs composePerms after sorting each bit, we apply the permutation for each bit to a larger vector. While this increases the communication required for the permutation application step, it does not add additional rounds and it allows us to eliminate the compose step and therefore

reduce the round complexity. We demonstrate analytically in Table 2 and empirically in Figure 11 that our protocol typically outperforms Asharov et al. [5] for both $\ell = 32$ and $\ell = 64$ bits. Because the codebase of Asharov et al. [5] is proprietary, we provide our own reimplementations of their protocol that we use in the Figure 11 benchmarks.

This may seem contradictory to the claims of Asharov et al. In fact, they claim that it is precisely the permutation composition step that leads to their improvement in performance over Bogdanov et al. The distinction is that the protocol of Asharov et al. has constant factors that scale better than Bogdanov et al. (and better than our protocol) as ℓ grows without bound, whereas our analysis focuses on the values $\ell = 32$ and 64 that we use in our shuffling and sorting frameworks.

Relationship to Bogdanov et al. [16]. Here we show that by instantiating the components of the Bogdanov et al. protocol with the more efficient variants proposed in recent works, we arrive at our protocol. Concretely, there are three differences in our protocols. First, their work converts the bit to sort into an arithmetic sharing and subsequently computes the $[[ord]]$ vector, whereas our work and Asharov et al. do the same using the $genBitPerm$ subprotocol. Second, they apply the $[[ord]]$ permutation to the input vector using an oblivious shuffle, open, and local application, which we do with our equivalent $applyElementwisePerm$ protocol (Protocol 4.2 of Asharov et al. in the three-party setting). Finally, the input padding and permutation extraction procedure described in Section 5 of Bogdanov et al. is equivalent to our procedure in §B.2 for padding and permutation extraction, plus our protocol is generalized to include descending-order sorting.

Performance analysis. In the remainder of this section and in Table 2, we compare the costs of our protocol and Asharov et al. [5], specifically in the three-party setting as that is the only one they support.

Common elements. Both protocols invoke $genBitPerm$ ℓ times. Each invocation incurs n calls to $b2abit$ and n multiplications that cost $3\ell_\sigma n$ and $\ell_\sigma n$ bits of communication, and 3 and 1 rounds of communication, respectively, with elements of size ℓ_σ since $genBitPerm$ computes a permutation. In total, each protocol communicates $4\ell_\sigma n$ bits over 4 rounds for each call to $genBitPerm$, amounting to $4\ell \cdot \ell_\sigma n$ bits of communication and 4ℓ rounds in total.

Costs of Asharov et al. There are two additional costs in Asharov et al. [5]: $\ell - 1$ calls to $applyElementwisePerm$ and $\ell - 1$ calls to $composePerms$. First, each call to $applyElementwisePerm$ operates over a single bit; using Table 1, we see that the total cost is $(\ell - 1)(6n + 7\ell_\sigma n)$ bits of communication over $7(\ell - 1)$ rounds. Second, the calls to $composePerms$ collectively cost $13(\ell - 1)\ell_\sigma n$ bits of communication and $7(\ell - 1)$ rounds of communication. When adding the cost of the ℓ calls to $genBitPerm$, the total cost

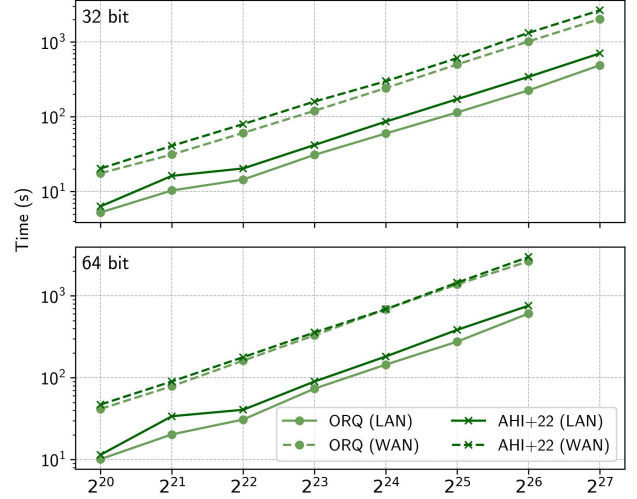


Figure 11. Comparison of our radixsort protocol with Asharov et al. [5] for (a) $\ell = 32$ and (b) $\ell = 64$. All data points are the average of 3 runs and are run with 34 threads in the 3-party setting. WAN data points are collected in a WAN environment with 20ms latency. 64-bit radixsort runs out of memory for 2^{27} input rows. Our hybrid protocol wins in all scenarios by up to 1.44×.

of the protocol is

$$24\ell \cdot \ell_\sigma n - 20\ell_\sigma n + 6(\ell - 1)n$$

bits of communication over $18\ell - 14$ rounds.

Our costs. In our protocol, we eliminate the permutation compositions and instead apply the permutations to a vector of bitwidth $\ell + \ell_\sigma$. We make:

- $\ell - 1$ calls to $applyElementwisePerm$, each costing $6(\ell + \ell_\sigma)n + 7\ell_\sigma n$ bits of communication and 7 rounds.
- One call to each of $convertElementwisePerm$ and $invertElementwisePerm$, adding $26\ell_\sigma n$ bits of communication and 14 rounds of communication.

Therefore, in total, our protocol has a cost of

$$17\ell \cdot \ell_\sigma n + 13\ell_\sigma n + 6\ell^2 n - 6\ell n$$

bits of communication and $11\ell + 7$ rounds of communication.

Discussion. In Table 2, we describe the costs both as a function of ℓ and ℓ_σ and for three values of the bitwidth. Our protocol has the better concrete round complexity, scaling as 11ℓ rather than 18ℓ (although they are asymptotically equivalent). For communication complexity, because ℓ and $\log n$ are often similar, the two protocols are typically asymptotically equivalent. Concretely, our protocol has the smaller constant factor for the $\ell \cdot \ell_\sigma$ term, which for typical values of ℓ close to ℓ_σ (e.g., $\ell = \ell_\sigma = 32$) means our protocol will require less communication. However, only our protocol has an ℓ^2 term, so it is a poor choice when $\ell \gg \ell_\sigma$.

	Asharov et al. (AHI+22)	Ours
Asymptotic Comm.	$O(\ell n \log n)$	$O(\ell^2 n)$
Concrete Comm.	$24\ell \cdot \ell_\sigma n - 20\ell_\sigma n + 6(\ell - 1)n$	$17\ell \cdot \ell_\sigma n + 13\ell_\sigma n + 6\ell^2 n - 6\ell n$
Asymptotic Rounds	$O(\ell)$	$O(\ell)$
Concrete Rounds	$18\ell - 14$	$11\ell + 7$
$\ell = 1$ Comm.	$128n$ (4 rounds)	$960n$ (18 rounds)
$\ell = 32$ Comm.	$24122n$ (562 rounds)	$23776n$ (359 rounds)
$\ell = 64$ Comm.	$48890n$ (1138 rounds)	$59424n$ (711 rounds)

Table 2. Cost analysis of the state-of-the-art radixsort protocol by Asharov et al. [5] and our hybrid radixsort protocol.

We discuss a few specific examples that are shown in Table 2 for the setting $\ell_\sigma = 32$ (since we never sort more than 4 billion elements due to speed constraints).

- If $\ell = 32$ as well, then our protocol saves a modest 1.4% improvement in total communication and a significant 36% improvement in communication rounds. In practice, the improved round complexity leads to substantially better performance, which we demonstrate empirically in both the LAN and WAN settings in Figure 11.
- If $\ell = 64$, then our protocol requires 22% more communication but decreases the number of rounds by 37%. Once again, Figure 11 shows empirically that ORQ performs better in both the LAN and WAN settings when $\ell = 64$, although by a slimmer margin than with $\ell = 32$.
- If $\ell = 1$ then the constant additive overhead of input padding and permutation extraction makes our protocol inferior to Asharov et al.

B.4 Bounding preprocessing for Quicksort

In the two-party setting, we need to generate Beaver triples for the quicksort comparisons in the preprocessing phase. Quicksort involves a nondeterministic number of comparisons, but we would like to probabilistically bound the number of comparisons so we can generate triples ahead of time. For n input elements, we generate Beaver triples for $2n \lg n$ comparisons, which is sufficient approximately 99.9% of the time. We make use of the work of McDiarmid et al. [54], which discusses this exact problem in the case of a random pivot selection. Since we shuffle the input list, our method of pivot selection chooses a random element, so the analysis applies.

The expected number of comparisons q_n for quicksort with uniformly random pivot selection as $n \rightarrow \infty$ is

$$\begin{aligned} q_n &= 2n \ln n - (4 - 2\gamma)n + 2 \ln n + O(1) \\ &\leq 2n \ln n = (\ln 4) n \lg n \leq 1.39n \lg n. \end{aligned}$$

This says nothing about how concentrated the distribution is around this expectation. Let Q_n be the random variable representing the actual number of comparisons required for a given run of quicksort. McDiarmid et al. [54] discuss the

probability $p = \Pr[|Q_n/q_n - 1| > \varepsilon]$ describing the concentration. Here, $(\varepsilon + 1)$ is the *multiplicative* overhead over the expectation q_n . We want to solve for ε as a function of both the input size n and the probability p . We set the probability to a constant $p = 2^{-10}$. This is significantly larger than a typical statistical failure, because if we “fail,” we can simply generate more triples in the online phase with no impact on security and an acceptable performance penalty. The chosen value of p makes our generated randomness sufficient for 99.9% of executions.

Theorem 1 of McDiarmid et al. [54] gives the following expression of the above probability.

$$p = n^{-2\varepsilon(\ln(\ln(n)) - \ln(\frac{1}{\varepsilon}) + O(\ln^{(3)}(n)))} \leq n^{-2\varepsilon(\ln(\ln(n)) - \ln(\frac{1}{\varepsilon}))}$$

To write ε as a function of p , we can take the natural log of both sides of this inequality. Even so, it is not feasible to calculate an analytic solution to this equation due to the presence of both ε and $\ln(1/\varepsilon)$ terms. To simplify the equation, we use the bound $\varepsilon \geq 0.43$ because $1.39 \cdot 1.43 \approx 2$. We simplify the equation using this value.

$$\varepsilon \leq -\frac{\ln(p)}{2 \ln(n)(\ln(\ln(n)) - \ln(\frac{1}{0.43}))}$$

For $p = 2^{-10}$, $\varepsilon \leq 0.43$ for all $n \geq 1300$, meaning the chosen value of $\varepsilon = 0.43$ is suitable for all $n \geq 1300$. To offset the fact that values of $n < 1300$ are not covered by this case (however unlikely it may be for us to need a sort that small in practice), we handle this case separately with an additive buffer of 10,000 triples whenever the input size is below $n = 2000$. This buffer is loose but sufficient, and it will not have a noticeable impact on performance for any reasonable use of the library.

C Analysis of the Join Operator

In this appendix, we address the security of our join operator and then describe its correctness in detail.

Security. Security comes naturally from our oblivious building blocks: since we never open any secret-shared data, and make no assumptions about the true cardinality of input or output tables, any execution of the join operator on tables L_1, R_1 is identically distributed to any other execution

on similarly-sized tables L_2, R_2 ; $|L_1| = |L_2|$ and $|R_1| = |R_2|$. Note that here we refer to the observable size of the table in memory (both rows and columns) and not the number of valid rows (which would be the “size” of the table in the conventional sense). For example, even if L_2 and R_2 consisted entirely of (secret-shared) zero values, an execution of the join protocol would be indistinguishable from an execution on real values.

In more detail, we can analyze the security of the join operator using the arithmetic black-box model (e.g., [25, §2.1]), which provides an ideal functionality abstraction of all primitive MPC protocols for secret-sharing and reconstructing secrets, performing arithmetic and boolean operations, and converting between arithmetic and boolean shares. Through inspection of Protocols 1, 2, and 3, we can see that the join operators in ORQ make use of MPC primitives in a black-box way. Concretely, the join-aggregation in Protocol 3 only uses oblivious logical operators (e.g., AND and NOT), (in)equality comparisons to find distinct keys, and calls to underlying methods like AGGNET, TABLESORT, and Concat. These methods, in turn, are shown in Protocols 1-3 to rely exclusively on oblivious comparisons and logical operations along with the oblivious shuffling and sorting protocols defined in Appendices A-B. Finally, inspection of all shuffling and sorting protocols (in Protocols 4-11) reveals that they only rely on oblivious arithmetic and boolean operators. Importantly, none of these algorithms opens sensitive data; the only openings are on permuted data (Protocols 5-6) or control flow data (Protocol 9) that have been carefully constructed so that the distribution of openings is independent of the data. As a result, ORQ join operators inherit the security guarantees of the underlying MPC protocols.

Correctness. In the remainder of this section, we argue the correctness of the inner join operation and its variants. We will use the following important fact when reasoning about joins:

Fact 2. *Invalid rows are never revalidated.*

This fact is guaranteed by only operating on the valid column using logical AND. Thus, rows may move from valid to invalid ($\text{Valid} \wedge 0 \rightarrow \text{Invalid}$) but once invalidated can never be revalidated ($\forall x : \text{Invalid} \wedge x \rightarrow \text{Invalid}$). Rows are only validated when creating a *fresh* table.

Theorem 1 (Informal). *The inner join protocol is correct.*

Proof Sketch. Define the key and valid columns of each input table as $L.k, L.V; R.k, R.V$, respectively. Without loss of generality we only consider a single abstract data column, $L.a$ and $R.b$, in each of the two tables. Assume the left table L has unique keys – each distinct value of k occurs at most once – and R has an arbitrary key distribution. Note that primary-key foreign-key relations are a strict subset of unique-key many-key relations, since keys on the right in a one-to-many relation need not exist on the left. Finally, while we discuss

the case of a single key k below, our protocol transparently handles compound keys, $k_1 || k_2 || \dots$, exactly as if all such key columns were concatenated into one.

The first step of the protocol is to concatenate the two tables. We concatenate by merging the schemas of the two tables and copying each table’s rows into the appropriate columns. All other values are zero (such as within left-table rows but under right-table columns). We additionally append a T_{id} column, which is 0 for left-table rows and 1 for right-table rows, and sort by $V || k || T_{id}$. Any invalid rows in either table would remain invalid after concatenation and thus be sorted to the top of the table. The resulting table $O = \text{CONCAT}(L, R)$ may have the form:

$O.k$	$O.a$	$O.b$	$O.V$	$O.T_{id}$
-	-	-	0	-
k_1	$L.a_1$	0	1	0
k_2	$L.a_2$	0	1	0
k_2	0	$R.b_1$	1	1
k_2	0	$R.b_2$	1	1
k_2	0	$R.b_3$	1	1
k_3	0	$R.b_4$	1	1

Next we discuss the two main functions of our inner join algorithm: (a) identifying matching rows, according to the join keys, and (b) copying data into the result of the join.

We first turn to the identification of matching rows. This step takes the form of invalidating rows which do not belong to the output of the join. In the case of an inner join, we only keep those (valid) rows from the right for which their key matches an associated unique key on the left:

$$\begin{aligned}
 O.V_i &:= O.V_i \wedge [O.T_{id,i} = 1] && \text{valid row from } R \\
 &\wedge [\exists j : O.k_j = O.k_i] && \text{exists matching key} \\
 &\wedge O.T_{id,j} = 0 && \text{from left} \\
 &\wedge O.V_j = 1] && \text{which is valid}
 \end{aligned}$$

Due to previously having sorted on $V || k || T_{id}$, we know that if any such $O.k_j$ exists, it will occur as the first position of this k -group. If not, a row from the right appears as the first position of this k -group, and no rows with this key are in the output of the join. We use the DISTINCT operator to mark the first row of each group with a 1. Then, the join output is formed by updating a temporary valid bit, $O.V_o := O.V \wedge \neg \text{DISTINCT}(V, k)$. That is, $O.V_o$ invalidates any valid rows which are the *first* of their (V, k) group.²

For each group four possibilities exist, shown in the tables below.

Case I. There are duplicate keys on the left, and no matching keys on the right. The validity update procedure marks

²We cannot directly update $O.V$, because it will later be used as an aggregation key, during the execution of which we propagate changes to $O.V_o$. However, changing aggregation keys during execution breaks correctness, so we keep a temporary column.

the first row from the left invalid, and aggregation (below) will mark the entire group invalid. This represents (possibly duplicated) primary keys with no foreign key rows.

Case II. There are duplicate rows on the left, and greater than zero matching keys on the right. The first left row is marked invalid, and aggregation similarly invalidates the entire left group. The right group remains valid.

Case III. This is another matching case, but with exactly one primary key on the left. The single row from the left is invalidated, but the aggregation makes no changes. Case III is frequently observed in realistic workloads, where our join operator operates on an explicit primary-key foreign-key relation.

Case IV. There are no matching rows on the left. Now we invalidate the *first row from the right*, and the subsequent aggregation invalidates *all rows on the right*. With no key on the left, this key is not be part of the output table.

$O.k$	$\neg \text{DIST}$	$O.V$
...	...	0
...	...	...
$L.k$	0	1
$L.k$	1	1
$L.k$	1	1
$L.k$	1	1

Table 3. Case I

$O.k$	$\neg \text{DIST}$	$O.V$
...	...	0
...	...	...
$L.k$	0	1
$L.k$	1	1
$R.k$	1	1
$R.k$	1	1

Table 4. Case II

$O.k$	$\neg \text{DIST}$	$O.V$
...	...	0
...	...	...
$L.k$	0	1
$R.k$	1	1
$R.k$	1	1
$R.k$	1	1

Table 5. Case III

$O.k$	$\neg \text{DIST}$	$O.V$
...	...	0
...	...	...
$R.k$	0	1
$R.k$	1	1
$R.k$	1	1
$R.k$	1	1

Table 6. Case IV

In Cases I and II, there are multiple matching “primary keys”; we operate as if only the first existed. We tend to avoid this mode of operation, since it does not follow the semantics of pk-fk relations, but it is useful in some environments, such as when performing semi-join. It also allows us to natively perform $\text{COUNT}(\text{DISTINCT}(\dots))$ operations over many-to-many joins (with duplicates on both sides).

After the invalid rows are marked, a final aggregation is applied (as part of the copy step, below) to similarly invalidate all rows in the group. This aggregation is a special case of the aggregation step outlined below, as the aggregation keys here are $V \parallel k \parallel T_{id}$. This allows for an invalidated left

row to invalidate all other left rows, and an invalidated right row to invalidate all other right rows, but invalidation will never cross the table boundary.

The size of the output of a one-to-many inner join is bounded by the size of the right table: assume every key on the right is present exactly once on the left. Then the output of the join is exactly the right table. Because our protocol is oblivious, it must be input-independent, even in the worst case. Thus our output must always exactly the size of the right table; we use the valid column to mark rows not actually present in the join output.

The cardinality equivalence means it is most efficient to build the output of the join within the rows of the right table. As such, all data from the right is included in the output by default. Data from the left table, on the other hand, must be explicitly copied. This operation is performed with a *group-by aggregation* and implemented via AGGNET. In practice, we have noticed that this default behavior also matches the semantics of many queries, so few copy aggregations are required.

Groups are defined as $V \parallel k$ (note the exclusion of the table ID column). The aggregation performed is a simple Mux, where the selection bit denotes whether values belong to the same group. We perform a reverse aggregation, which has the effect of aggregating *down*, and copying the first row of each multiplex group into all other rows of that group. Where a row on the left exists, it is the first row, due to the final sort by T_{id} ; should multiple left rows exist, we only copy the value from the first. All left rows will be invalidated (Case II, above). The join operator does not support, at this stage, many-to-many-key joins. However, with a prior aggregation, or under certain conditions (e.g., the join performs a $\text{DISTINCT}(\cdot)$ aggregation), our operator still applies. See Section 3.3 and 3.6 for more details. $\square$

Unifying join and aggregation. The validity-update procedure of join looks very similar to aggregations performed over joined tables. For improved efficiency, we would like to combine these two operations into a single unified control flow, which will approximately halve the number of operations required.

Theorem 2 (Informal). *Aggregations can be performed in the same control flow as the join operator provided the following conditions are met:*

1. The aggregation keys are the same as the join keys.
2. No aggregation key is also an aggregation output.
3. No aggregation input is also an aggregation output for a different aggregation.

The intuition behind this theorem is that the conditions preclude any interaction between individual aggregations, so they can be evaluated in parallel: calling $\text{AGGNET}(f_1, \dots)$, $\text{AGGNET}(f_2, \dots)$, $\text{AGGNET}(f_3, \dots)$ is equivalent to

$$\text{AGGNET}(\{f_1, f_2, f_3, \dots\})$$

If any of the conditions are not true, these calls are *not* independent (and may have side effects), so sequential calls to the operator are required.

Handling other scenarios. Our operator only provides correct semantics for the case where there are at most unique keys on the left. To handle duplicate keys, we must first pre-aggregate over the left key. This requires an additional call to AGGNET, after which the aggregated table has unique keys and aggregated values. Then, our operator can be applied. We note that this collapsing is only valid for self-decomposable aggregation functions.

This general technique is also applicable, as alluded to above, if one of the join or aggregation keys are a prefix of the other. (i.e., $K_j = K_a || K^*$ or $K_a = K_j || K^*$). Then, we sort on the longest combined key ($\text{argmax}(|K_j|, |K_a|)$) and perform the join normally. If the aggregation keys are a prefix of the join keys, we need to handle the possible interspersed rows from the left ($T_{id} = 0$): AGGNET requires all rows in a group to be adjacent. We can accomplish this by masking rows from the left from an identity element for the given aggregation (e.g. for sum, 0; for min, ∞). Then, while the join occurs over $K_j = K_a || K^*$, we can still aggregate over K_a by ignoring all K^* keys and taking advantage of the masked identity elements.

Finally, if the aggregation is applied to columns from both the left and the right (as in the Secure Yannakakis query, [71]) we must first break the aggregation function apart, apply either of the techniques above, and post-aggregate over the joined table. In the Secure Yannakakis example, we have:

```
1 SELECT T.class, SUM(S.cost * (1 - R.coinsurance))
2 FROM R, S, T
3 WHERE R.person=S.person AND S.disease=T.disease
4 GROUP BY T.class;
```

We observe that we can first pre-aggregate the quantity $1 - R.coinsurance$ per person, join $R \bowtie S$ on person, compute the product with cost, and then post-aggregate by summing over disease and class. This rewriting applies because the SUM function is self-decomposable.

Custom Aggregators. ORQ users can also define custom aggregation functions, as in the following example that computes a product:

```
template<typename A>
A prod(const A& a, const A& b) {
    return a * b; // User-defined logic here
}
```

ORQ will call $\text{Mux}(g, a, \text{prod}(a, b))$ within the aggregation control flow, where g denotes whether the elements belong to the same group and a, b are slices of the vector to be aggregated. For each pair of elements in the same group ($g_i = 1$), the aggregation function prod is applied; otherwise, no update occurs. For the example above, this will result in obviously multiplying all elements of each group.

Unique-key joins. An additional optimization is possible in the case of unique keys on both sides of the join. (We assume this information is included in the tables' public schema.) In this case, the call to AGGNET is unnecessary: since each row on the left has at most one match on the right, the call to DISTINCT is sufficient for computing the join; the output of is then bounded by $\min(|L|, |R|)$ rather than just $|R|$. Additionally, in this context, aggregating over the join keys is a nonsensical operation, so we omit it. This operation is now effectively just a PSI protocol; many more optimizations are possible. However, we only use this optimized join algorithm for the comparison with SecretFlow, since they only implement a PSI-join which requires unique keys.

C.1 Correctness of other types of joins

Outer joins. Outer joins work similarly to an inner join but with different validation rules. That is,

- A **left-outer join** is an inner join, plus all rows from the left.
- A **right-outer join** is an inner join, plus all rows from the right.
- A **full-outer join** contains all rows from both input tables, and is effectively just a concatenation with aggregation.

Full-outer join is the simplest case. Since we take all rows, there is nothing new to invalidate. We copy the valid bits from each input table and apply them to the joined output, which is just a concatenation $L || R$. (We may also sort, so that user-defined aggregations can be applied, if necessary.)

For a right-outer join, we do not invalidate any rows from the right. But by the semantics of our join operator, we only include new columns from the left if explicitly specified by the user with a copy aggregation. Any rows *originally* from the left table should always be removed. This validation rule, therefore, is also quite simple: $O.V \leftarrow O.V_{LR} \wedge O.T_{id}$, where rows from the left have $T_{id} = 0$.

To compute a left outer join, we can compute the inner join but should not invalidate any left-table rows. The desired relation is shown in the table below:

V_{LR}	T_{id}	DISTINCT	V
0	X	X	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

$$O.V \leftarrow O.V_{LR} \wedge \neg(O.T_{id} \wedge \text{DISTINCT}(K))$$

That is, invalidate rows from R which are the first of their group. When this occurs, it means no such row from L exists, so this row is not in the output relation. To only keep *unique*

(rather than all) rows from L , the second clause could instead be replaced with $O.Tid \oplus \text{DISTINCT}(K)$.

Semi-join. The semi-join $L \bowtie R$ returns all rows in L which match a row in R . Observe that we can reframe this operator into one we have already seen by swapping the tables: which operator returns all rows in R which match a row in L ? This is the inner join $R \bowtie L$. So, to implement $L \bowtie R$, we simply call $R \bowtie L$, and then project the result columns to maintain the semantics of semi-join (namely, only return columns in L). This final operation requires no extra work because our inner-join operator, by default, returns all data in columns from the “right” (here, L). We acknowledge that this transformation of semi-join to inner join is not correct in general, but rather a byproduct of the way our oblivious control flow is implemented for one-to-many joins.

To see why this rewriting works, consider what rows are removed from $R \bowtie L$ by the validity-update procedure of our inner join protocol: all rows from R , and all rows from L which do not have a matching “primary key” in R . Of course, in a semi-join, table R is not part of the output relation, so we should always remove all rows from R . And, if a row in L does not have a match in R , then it should be removed from the output, by the definition of semi-join.

Anti-join. The inverse of semi-join is anti-join, $L \not\bowtie R$, which removes from L all rows matching one in R . We apply a similar transformation: we need an operator which will ignore all rows from R , and keep rows from L which do not match any in R . A partial answer is the *right-outer join* from above, again applied in the opposite direction, $R \bowtie_{\text{right}} L$. This would keep all rows in the “right” (here, L), but it would also incorrectly keep rows from L which matched one from R .

The solution here is to apply a special aggregation over the valid bit³ which copies the valid bit of the *first* row of a group to all other rows in the group. We do not include $O.Tid$ as an aggregation key. Thus, we invalidate all rows for which the first row in that group was previously invalidated by the (right outer) join. Since this join will invalidate all “left” ($= R$) rows, any group for which a row exists in R will have its *first* row invalidated by the join, and in the subsequent aggregation, all such rows from this group will be invalidated. This correctly implements anti-join; to match the semantics of the operator, we again project to the columns on the left. Thanks to our unified join-aggregation control flow, this validation update is performed at the same time as the join, and incurs the same cost as a copy aggregation (i.e., $O(n \log n)$ multiplex operations) in a normal join. Again, we acknowledge that this rewriting of anti-join to right-outer join is not correct in relational algebra, but a convenient optimization in our setting.

One side effect of this rewriting (for both semi-join and anti-join) is that our implementations of these operators

transparently handles duplicates in R , by Cases I and II, above. Both semi-join and anti-join only consider the *existence*, in table R , of keys in L , so we need no special handling for duplicate keys.

C.2 Proof Sketch of AGGNET correctness

In this section, we provide an argument for why the AGGNET protocol achieves correctness. As an illustrative example, consider a table with $n = 2^l$ rows and three columns: a key column K , input column A , and output column G . We use these parameters for convenience but emphasize that the ideas generalize to any database. Additionally, while the aggregation network requires that the *full* table size is a power of two, each key can have arbitrary cardinality. If the number of input rows is not a power of two, we pad the table with dummy (invalid) rows and remove them after the protocol terminates.

The first step of the algorithm is to copy column A into column G (Line 1), since the aggregation function is applied in place. From this point, we only consider column G .

Claim 1. *Values across distinct keys are never aggregated.*

Take an arbitrary row i and distance d . Assume the keys in rows $(i, i + d)$ do not match. Then $b \leftarrow 0$ (Line 4) and we update $G_{i+d} \leftarrow \text{Mux}(0, G_{i+d}, g) = G_{i+d}$ (Line 6), so this row is not modified. Therefore, we only aggregate rows with matching keys.

Claim 2. *The algorithm correctly aggregates groups.*

AGGNET expects an input table sorted on the key column. This puts all rows of a given key-group next to each other. Assume row j is the final row of some arbitrary group k of size $m \leq n$. Then, we want to argue that the final value $G_j = f(G_{j-m}, \dots, G_j)$.

In the final round of the protocol, G_j is updated with the value $n/2$ indices earlier (assume for the sake of argument that $m > n/2$), $G_j \leftarrow f(G_{j-n/2}, G_j)$. In the prior round, each of these two values were updated with their own value along with the values $n/4$ indices prior. The algorithm proceeds in this manner, taking the next power of two in each round. In the first round we have a distance $d = 1$ so adjacent elements are aggregated.

This procedure is easy to visualize if we consider this aggregation structure as a tree, where each “prior index” is a left branch, and the “same index” is a right branch. AGGNET builds a binary tree, where the path through the tree from G_j to G_{j-h} , for arbitrary $h \leq m$, corresponds to the binary representation of h . Each leaf node has a single unique path to G_j , since each offset h has a unique binary representation. Thus, each input value is aggregated once and in order of its appearance in the original list. Since the aggregation function f is self-decomposable, we have that $f(f(X), f(Y)) = f(X, Y)$, so by induction we can show that the final result is $f(G_{j-m}, \dots, G_j)$. $\square$

³Specifically, this is just a copy aggregation, which already has special handling in order to copy rows from the left to right.

C.3 Trimming the output of Join operations

In Section 3.3, we discussed bounding the outputs of joins to the size of the right table. Here, we elaborate on the heuristic used to automatically govern this trim operation. It is important that, due to the obliviousness of our operators, the “size” of tables discussed in this section refers to the secret-shared table size, *not* the number of valid rows (which is not known at runtime).

First, we provide intuition by way of an example. Consider a join between two asymmetric tables, $|L| = 10$ and $|R| = 10^6$. After join, we have a larger table of size $|O| = 10 + 10^6$. Clearly it is not worth trimming: we must perform valid-bit sort on a table of size $|O| \approx |R| = 10^6$, but are only able to remove $|L| = 10$ elements.

However, if a join is more symmetric, say $|L| = 10^6$ and $|R| = 2 \cdot 10^6$, trimming is much more promising. While we sort on $|O| = 3 \cdot 10^6$ elements, we can then remove 1/3 of the output rows.

Trimming brings significant improvements in composed queries where we also perform aggregations on the output of the join. Since the odd-even aggregation algorithm requires power-of-two-sized inputs, trimming frequently allows us to “drop” below the next power of two, leading to significant speedups in query execution. In the analysis below, we optimize for such composed operation, assuming a join will be followed by additional joins with other tables.

To establish the heuristic, we define the abstract costs of each of our major operators. Let $J(n)$, $V(n)$, $S(n)$, and $A(n)$ be the respective costs of join, valid-bit sort, full sort, and aggregation over n rows (we abstract away aggregation functions at this phase).

In this analysis, we will say we first join two tables of size L and R , with $N := L + R$ the combined size. A subsequent join is performed with a table of size T . Intuitively, we want to trim if the additional cost (valid-bit sorting over N rows) is less than the future savings. Expressed formally,

$$J(N + T) > V(N) + J(R + T) \implies \text{should trim}$$

First, we observe that joins are superlinear: $\forall n, m > 0 : J(n + m) \geq J(n) + J(m)$. This follows from both the superlinearity of both sorting and aggregation.

$$\begin{aligned} J(N + T) &> J(L) + J(R + T) \\ J(L) + J(R + T) &> V(N) + J(R + T) \\ J(L) &> V(N) \implies \text{trim} \end{aligned}$$

That is, we should trim when the cost of performing a join over the trimmed rows would be more expensive than performing a valid-bit sort over the entire table.

We now decompose each operator:

- Valid-bit sorting is implemented with radixsort so is approximately linear, $V(n + m) \approx V(n) + V(m)$.

- Joins consist of a two single-bit sorts (valid column plus T_{id}), a regular sort, and an aggregation; $J(n) \approx 2V(n) + S(n) + A(n)$.

Define $\alpha := R/L$. For most applications involving unique-key joins, $\alpha > 1$. Then,

$$\begin{aligned} V(N) &< J(L) \implies \text{trim} \\ V(L) + V(R) &< 2V(L) + S(L) + A(L) \\ (\alpha - 1)V(L) &< S(L) + A(L) \end{aligned}$$

It is difficult to reason, in general, about the cost of aggregation, $A(\cdot)$, since this cost depends heavily on (possibly user-defined) aggregation functions. However, it can be bounded from below considering only the control flow of the aggregation operator itself. Let ω be the bitwidth of values in the table. Then both aggregation and quicksort perform at least $n \lg(n)$ comparisons,⁴ which, under MPC, each require $\log \omega$ operations. Valid-bit sorting, on the other hand, performs $6n$ ω -bit permutation operations, each of which we can approximate as costing N multiplications, where N is the number of parties required by the MPC protocol. Inserting these costs into the above, we see that:

$$\begin{aligned} (\alpha - 1)6NL &< 2L \lg(L) \lg(\omega) \implies \text{trim} \\ 3\alpha N &< \lg(L) \lg(\omega) \end{aligned}$$

For our common use cases, $\omega = 128$ bits (due to input padding) and $N \in \{2, 3, 4\}$. The table below gives some example sizes, and confirms our intuitions about only trimming when we can remove sufficiently many rows from the left table.

L	α for $N = 3\text{PC}$	Trim when $R < \dots$
100	5.2	516
10k	10.3	103k
1M	15.5	15.5M
100M	20.7	2.07B

This analysis differs slightly from the version of the heuristic we have implemented (which was originally derived for radixsort, not quicksort). However, the concrete decisions it makes in real queries are similar.

D Bandwidth Measurements

In Table 7, we provide the bandwidth usage for each query used in the paper. Since TPC-H queries have a wide variety of input sizes (nearly an order-of-magnitude spread at a fixed Scale Factor), we report bandwidth usage in kilobytes per row. While these results have been collected at SF10, query execution scales only slightly superlinearly, so the values below provide a good bandwidth estimate across all query sizes reported in ORQ. Measurements are also normalized per party; some protocols are not perfectly symmetric, so we divide the *total communication* by the number of computing parties (2 in SH-DM, 3 in SH-HM, and 4 in Mal-HM, respectively). We observe that SH-DM has just under twice the

⁴Assuming randomized quicksort, this is true w.h.p.

Query	SH-DM	SH-HM	Mal-HM
TPC-H Q1	33.1	18.5	51.4
Q2	65.0	35.2	100.2
Q3	42.2	23.6	65.4
Q4	44.1	24.3	68.5
Q5	122.5	66.4	188.7
Q6	0.2	0.3	0.6
Q7	111.9	61.0	173.4
Q8	120.2	65.5	186.6
Q9	113.4	62.5	176.1
Q10	73.6	40.5	114.7
Q11	41.8	23.4	65.7
Q12	46.9	25.5	72.0
Q13	45.6	25.6	70.4
Q14	18.0	9.9	27.9
Q15	54.1	30.0	83.6
Q16	94.2	51.3	144.3
Q17	53.0	29.4	82.8
Q18	86.6	47.5	133.3
Q19	21.2	11.5	32.8
Q20	82.1	45.8	127.0
Q21	160.2	87.0	245.8
Q22	16.6	9.3	26.2
Aspirin	39.9	21.2	61.2
Comorbidity	42.5	23.5	65.8
Credit	26.6	14.0	40.6
SYan	42.2	23.9	65.6
Patients	22.3	12.0	34.5
Market Share	17.8	10.0	27.3
Password	26.9	14.8	40.8
C. Diff	27.4	15.0	41.6
Secrecy Q2	32.7	18.4	50.6

Table 7. Bandwidth (KB) per row and computing party, for all queries and protocols used in ORQ.

communication complexity of SH-HM, and Mal-HM about three times. Bandwidth for other experiments (§5.3) are provided in Tables 8, 9, 10, and 11.

E Performance in a geodistributed setting

In this section, we evaluate ORQ’s ability to amortize network costs in a geodistributed setting. We create a deployment spanning four AWS regions: us-east-1 (N.Virginia), us-east-2 (Ohio), us-west-1 (N.California), and us-west-2 (Oregon). The deployment is constrained by minimum link bandwidth between 4.23 – 8.47 Gbps, and RTT between 50 – 61 ms. To make the setting challenging, we deploy parties so that at least one link exists between the U.S. west and east coasts.

We show results for five TPC-H queries: two of the fastest (Q11, Q21), the median (Q12), and the two slowest (Q8, Q21). These five queries account for approximately 30% of the total

Query	Secrecy	ORQ
TPCH-Q6	0.3	0.3
Password	30.9	10.7
Credit	39.2	9.3
Comorbidity	56.4	23.7
C. Diff	63.1	12.1
Aspirin	2 820	10.7
TPCH-Q4	4 825	16.5
TPCH-Q13	1 712	20.0

Table 8. Bandwidth (KB) per row and party for the queries used in Fig. 5 (left). Secrecy’s high bandwidth is due to its $O(n^2)$ join operator and $O(n \log^2 n)$ sorting algorithm (bitonic sort).

Query	SecretFlow	ORQ
S1	<1	14
S2	<1	28
S3	88	7 677
S4	286	7 735
S5	2 835	29 834

Table 9. Bandwidth (bytes) per row and party for the queries used in Fig. 5 (right). SecretFlow’s low bandwidth is due to its join operator, which leaks matching rows to parties and allows them to perform most subsequent operations locally (i.e., without communication).

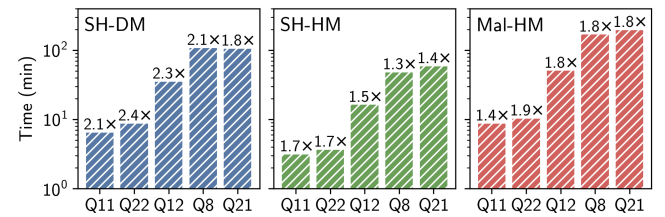


Figure 12. TPC-H query times at SF1 in a geo-distributed WAN deployment, with ratios over times in our symmetric WAN.

time to run the entire TPC-H benchmark. Figure 12 shows the execution time, along with the overhead compared to the symmetric WAN environment used in the experiments of Fig. 4. Latency increases by up to 2.4× for SH-DM, 1.7× for SH-HM, and 1.9× for Mal-HM. These results demonstrate that ORQ’s overhead is lower than the corresponding 3× increase in RTT between the two WAN environments, indicating its ability to effectively amortize network costs and achieve competitive performance.

Input Size	SecretFlow: SBK	ORQ: RS (64b)	SecretFlow: SBK_valid	ORQ: RS (32b)
100k	921.4	517.6	282.0	210.0
1M	9 212.4	5 176.0	3 212.2	2 100.0
10M	92 118.9	51 760.0	37 568.5	21 000.0

Table 10. Total bandwidth (MB) for SecretFlow and ORQ in the experiment of Fig. 6. ORQ’s optimized radixsort achieves between 1.34× and 1.79× lower bandwidth than the SecretFlow algorithms.

$k =$	SH-DM (2PC)		SH-HM (3PC)		Mal-HM (4PC)	
	MP-SPDZ	ORQ	MP-SPDZ	ORQ	MP-SPDZ	ORQ
10	165.3	5.3	18.7	11.9	272.0	40.3
11	363.7	10.6	37.4	23.9	600.4	80.6
12	793.4	21.2	74.8	47.8	1 315.3	161.3
13	1 719.0	42.4	149.6	95.6	2 860.5	322.7
14	3 702.2	84.8	299.2	191.2	6 182.2	645.5
15	7 933.0	169.6	598.4	382.4	13 288.1	1 291.0
16	16 922.9	339.2	1 196.7	764.9	28 424.6	2 582.1
17	35 959.8	678.4	2 393.5	1 529.8	60 547.2	5 164.2
18	76 147.6	1 356.8	4 787.1	3 059.7	128 492.2	10 328.4
19	160 751.0	2 713.7	9 574.2	6 119.4	271 780.4	20 656.9
20	338 414.0	5 427.4	19 148.4	12 238.9	(OOM)	
21	710 650.0	10 854.8	38 296.8	24 477.9		
22	(Crash)		76 593.9	48 955.9		
23			153 187.5	97 911.8		
24			306 375.0	195 823.6		

Table 11. Total bandwidth (MB) for MP-SPDZ and ORQ in the experiments of Fig. 7 (oblivious radixsort on 2^k rows). ORQ’s optimized radixsort achieves between 1.57× and 65.5× lower bandwidth than the MP-SPDZ algorithm. MP-SPDZ crashes at 2^{22} in SH-DM and runs out of memory at 2^{25} in SH-HM and 2^{20} in Mal-HM.